

天津大学

《软件工程中级实践》实验报告

题目：积分系统



学 院 软件学院
专 业 软件工程
年 级 2023级
小 组 第04小组

学号	姓名
3023244157	戴茗静
3023244158	曾意
3023208045	高灿
3023244362	杨晓越
3023244168	贾思韵

2025 年 12 月 12 日

目 录

第一章 模块需求说明	1
1.1 引言	1
1.1.1 简介	1
1.1.2 背景	1
1.1.3 定义、缩略语	2
1.2 需求分析	2
1.2.1 积分获取	2
1.2.2 积分消费	3
1.2.3 积分账户与有效期管理	3
1.2.4 规则配置与管理	4
1.2.5 系统集成与性能	4
1.3 业务分析	5
1.3.1 积分系统核心类类图	5
1.3.2 业务流程分析 (UML活动图)	6
1.3.3 积分系统用例图	9
第二章 模块设计	17
2.1 数据库设计	17
2.1.1 DB 一览表（新增部分）	17
2.1.2 表结构	17
2.2 后端接口设计	25
2.2.1 新增接口	25
2.3 前端设计	32
2.3.1 用户端	32
2.3.2 管理端	36
第三章 模块实现	39
3.1 整体架构设计	39
3.2 业务逻辑实现	39

3.2.1 积分系统核心模块的详细实现	39
3.2.2 营销系统积分计算的技术实现	49
3.2.3 RabbitMQ 异步消息处理的实现	52
3.2.4 Redis 缓存优化的实现	54
3.2.5 积分过期处理与预警的定时任务实现	56
3.2.6 数据完整性保障的多层验证机制	58
第四章 模块测试	59
4.1 测试概览	59
4.1.1 测试工具	59
4.1.2 测试范围	59
4.2 前端测试	60
4.2.1 页面加载与渲染测试（顾客端）	60
4.2.2 促销商品功能测试	61
4.2.3 积分抵扣支付功能测试	61
4.2.4 积分兑换商品功能测试	62
4.2.5 积分抽奖功能测试	62
4.2.6 页面加载与渲染测试（管理端）	63
4.2.7 规则管理功能测试	64
4.2.8 预警配置功能测试（管理端）	64
4.2.9 兑换管理功能测试（管理端）	65
4.3 后端测试	65
4.3.1 单元测试	65
4.3.2 集成测试	67
4.3.3 API测试	73
4.3.4 数据完整性测试	75
4.4 前后端联合测试	76
4.4.1 用户积分全流程测试	76
4.4.2 积分支付流程完整测试	77
4.4.3 订单积分奖励流程测试	77

4.4.4 积分抽奖全流程测试	78
4.4.5 促销规则动态展示测试	78
4.4.6 积分过期预警流程测试	79
4.4.7 管理端规则配置同步测试	79
第五章 问题与解决&结果反思	81
5.1 问题与解决	81
5.2 结果反思	83
第六章 实践总结与展望	84
6.1 实践总结	84
6.2 未来展望	85

第一章 模块需求说明

1.1 引言

1.1.1 简介

本文档面向“饿了么”APP项目的客户决策者、项目经理、设计与开发团队、测试人员及相关参与方，旨在系统阐述产品新增的“积分系统”核心功能需求。

“饿了么”APP项目致力于提供高效、便捷的在线订餐和商户管理服务。为增强用户粘性、提升消费活跃度并构建完善的用户忠诚度体系，本项目新增“积分系统”功能。

“积分系统”的实现将通过多样化的积分获取途径和灵活的积分消费方式，建立正向的用户激励循环，有效提升用户复购率，并为精准营销和个性化促销提供数据支撑。

1.1.2 背景

(1)积分系统的必要性

成熟的电商和外卖平台普遍建立积分体系作为用户忠诚度计划的核心组成部分。原系统缺乏统一的积分管理机制，无法有效激励用户持续消费和参与平台互动，导致以下核心业务需求无法满足：

- a. 缺乏用户激励手段：无法通过积分奖励鼓励用户完成消费、互动等行为，用户粘性提升受限。
- b. 促销活动单一：缺少基于积分的灵活促销规则，难以实现精准营销和个性化优惠。
- c. 会员体系不完善：会员权益缺乏实质性的积分支撑，会员价值感知度低。
- d. 用户行为数据缺失：无法通过积分变动追踪用户行为偏好，数据驱动决策能力不足。

(2)项目核心价值提升

积分系统作为用户忠诚度体系的核心，用于量化和管理用户在平台内的价值积累，其增加将带来以下核心价值：

- a. 构建完整用户激励体系：通过积分获取和消费的双向机制，建立正向的用户行为激励循环。

b. 增强营销活动灵活性：支持多种积分规则配置，实现节假日促销、会员专属活动等精准营销。

c. 提升用户粘性和活跃度：积分过期机制和多样化消费场景促进用户持续参与和消费。

d. 丰富会员权益内涵：不同会员等级享有差异化积分权益，增强会员身份的价值感知。

1.1.3 定义、缩略语

表 1-1 定义与缩略语

术语	解释
积分系统	指平台的积分管理体系，用户可以通过消费、促销活动、会员升级等行为获得积分，并用于支付抵扣或商品兑换
积分交易	用户积分账户的变动行为，包括积分获得、积分消费、积分过期等；交易类型分为收入、支出和过期
积分规则	管理积分获取和消费的业务规则，包括基础消费比例、促销倍数、会员奖励等可配置参数
会员等级	用户通过升级成为不同等级的会员，等级越高的会员在积分获取比例、专属活动等方面享有更多权益
积分兑换	用户使用积分直接兑换商品的功能

1.2 需求分析

积分系统是外卖平台营销体系的核心组成部分，旨在通过积分激励提升用户活跃度、复购率和平台粘性。系统需要支持积分的获取、消费、查询、过期管理以及规则配置等功能，并确保数据准确性、业务灵活性和系统高性能。

1.2.1 积分获取

用户可通过以下途径获取积分：

- 基础消费积分：用户下单后，根据订单金额和对应会员等级的积分比例计算奖励积分。使用积分兑换商品不重复获得基础积分。初步制定的基础消费积分获取规则为：普通用户消费1元获得0.1积分；白银会员消费1元获得0.5积分；黄金会员消费1元获得1积分；钻石会员消费1元获得1.5积分，积分有效期为30天。

- 促销积分：在基础积分之上，按促销规则叠加倍数积分。支持多种规则（如节假日x倍、满额x倍、指定商品x倍、金额区间x倍及其组合），同一订单仅应用优先级最高的一条规则。初步制定的促销积分获取规则为：十一黄金周消费获得双倍积分奖励；
- 钻石会员消费满200元获得双倍积分奖励；购买指定商品1.5倍积分奖励；双十二会员消费满500获得双倍积分奖励.....积分有效期为30天。促销积分获取规则以轮播图的形式在首页展示给用户，若涉及到指定商品，在该轮播图上添加“立即购买”入口，点击即可查看所有指定商品，若选择购买，则进入该商品的支付界面。
- 等级积分：用户成为或升级会员时，一次性发放与会员等级对应的固定积分奖励。初步制定的等级积分获取规则为：成为白银会员奖励100积分；成为黄金会员奖励200积分；成为钻石会员奖励300积分，积分有效期为30天。
- 行为积分：用户首次点赞店铺、收藏店铺或还贷款时，按会员等级发放固定积分，同一店铺的重复行为不重复奖励；会员可参与抽奖活动：白银会员每月有1次抽奖机会，黄金会员每月有2次抽奖机会，钻石会员每月有3次抽奖机会，不同会员对应的奖池和中奖概率不同，以白银会员为例，奖池包含：奖励50积分（20%概率）、奖励100积分（10%概率）、奖励150积分（5%概率）、积分翻倍（5%概率）、未中奖（60%概率），积分有效期为15天。

1.2.2 积分消费

- 积分+现金支付：支付时自动计算可用积分可抵扣的现金金额（默认支付方式），用户也可选择纯现金支付。前端需实时展示抵扣金额与优惠信息。初步设置的兑换比例：10积分=1元。
- 积分兑换商品：用户直接使用积分兑换指定商品（不经过购物车），需选择配送地址。兑换订单不可取消，积分不可退回，且受商品库存限制。

1.2.3 积分账户与有效期管理

- 账户结构：维护总积分、可用积分、冻结积分，确保数据一致性且不允许出现负数。
- 积分过期：积分设置有效期，通过定时任务过期自动扣除；支持过期前预警（如提前3天发送消息），预警规则可配置。
- 积分明细：用户可查询积分变动记录，支持按交易类型（获得、消费、过期）和积分来源筛选，并展示描述信息。

1.2.4 规则配置与管理

- 动态规则管理：管理员可通过接口对积分规则（基础比例、促销倍数、行为奖励、等级奖励等）、兑换规则（积分兑现金比例、商品兑换所需积分及库存）和积分抽奖规则进行增删改查，实现业务灵活调整。
- 优先级控制：促销规则支持优先级配置，确保多条规则触发时按预期应用。

1.2.5 系统集成与性能

- 系统集成：与订单系统、虚拟钱包、营销系统、商家互动（点赞、收藏）等模块对接。订单支付后通过异步消息（RabbitMQ消息队列）触发积分发放；其余业务场景使用同步模式：订单状态变更（取消、完成）时，同步处理积分冻结、解冻、扣除或回退。
- 性能与容错：使用Redis缓存积分账户、兑换比例等热点数据，采用Cache-Aside模式提升查询性能；写操作后删除缓存，保障数据一致性；支持高并发场景，通过数据库行锁与事务确保积分操作原子性；Redis故障时降级至数据库查询，保证核心功能可用。

1.3 业务分析

1.3.1 积分系统核心类类图

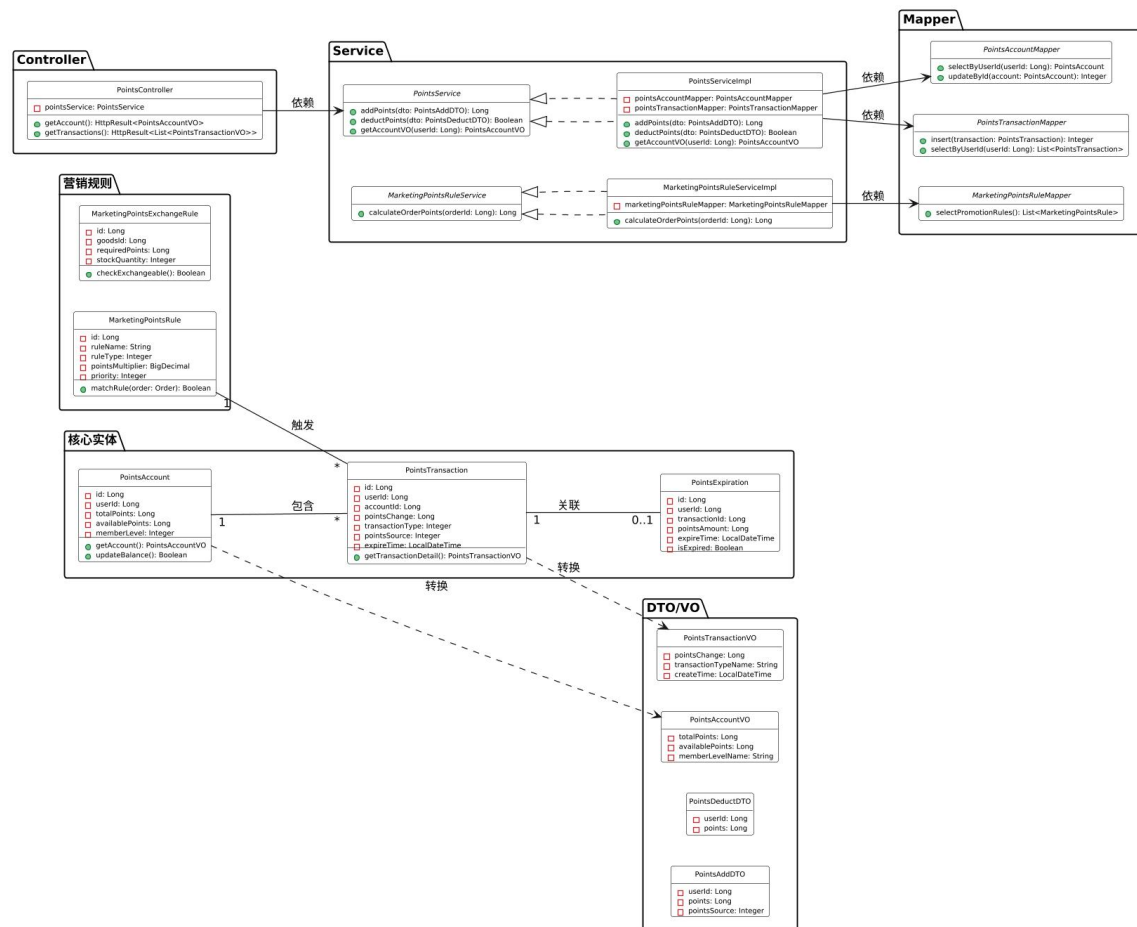


图 1-2 积分系统核心类类图

1.3.2 业务流程分析 (UML活动图)

(1) 用户获取积分活动图

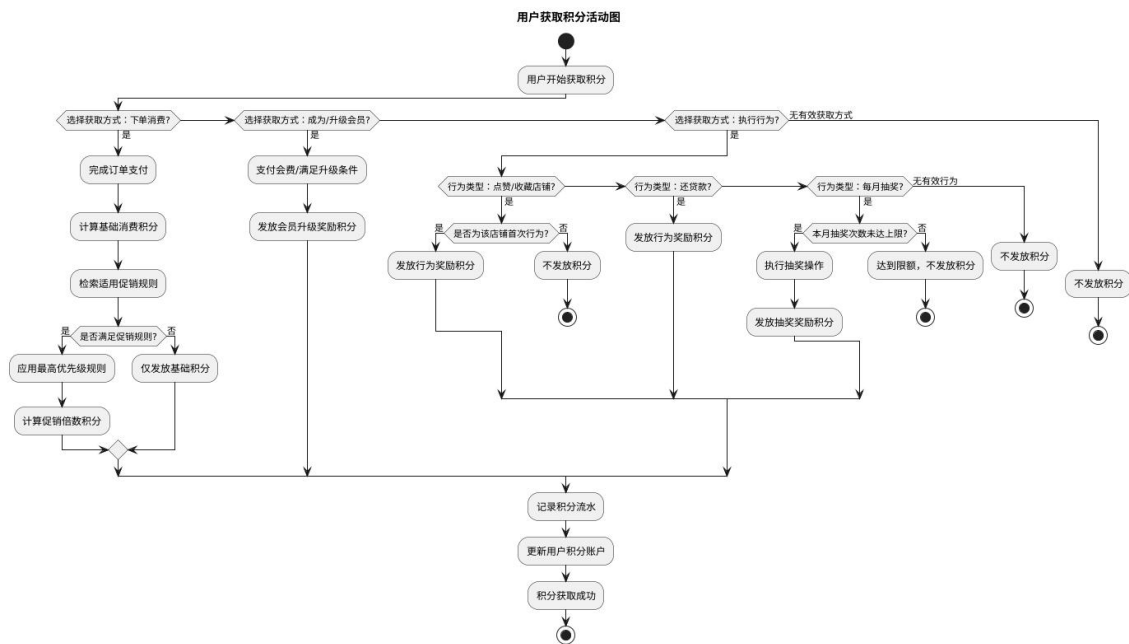


图 1-3 用户获取积分活动图

(2) 用户消费积分活动图

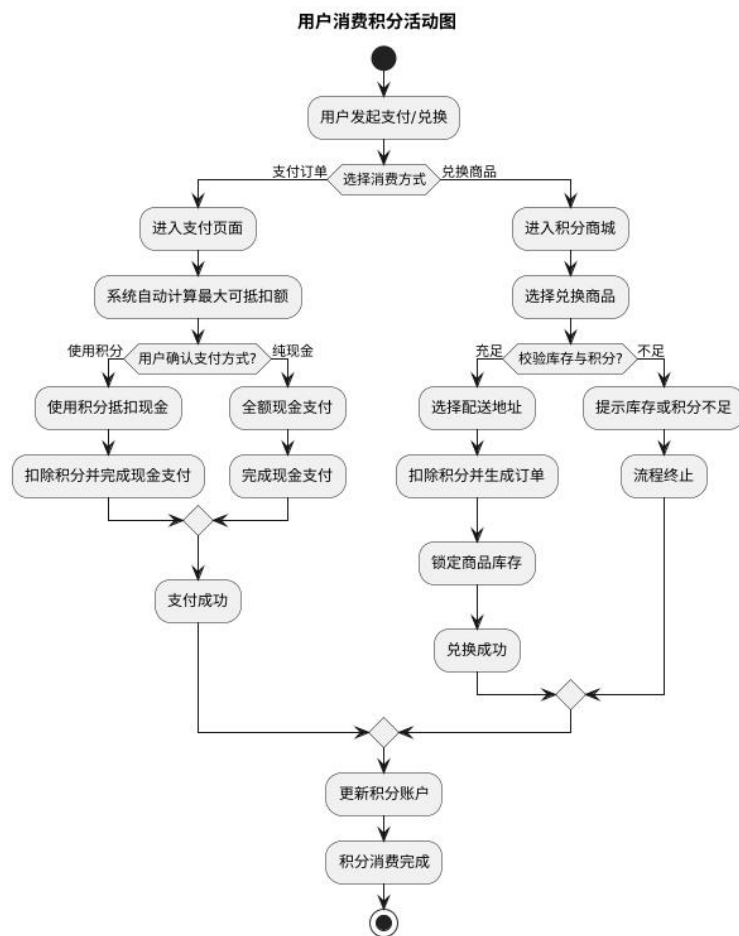


图 1-4 用户消费积分活动图

(3) 系统自动化处理活动图

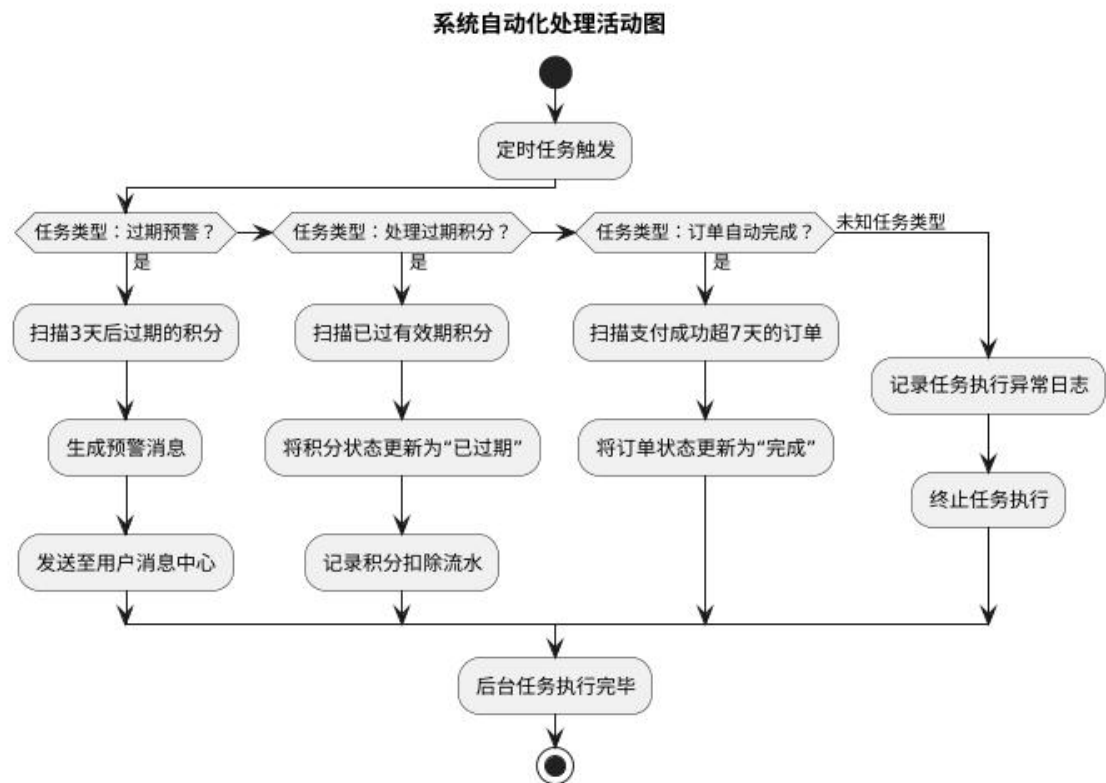


图 1-5 系统自动化处理活动图

1.3.3 积分系统用例图

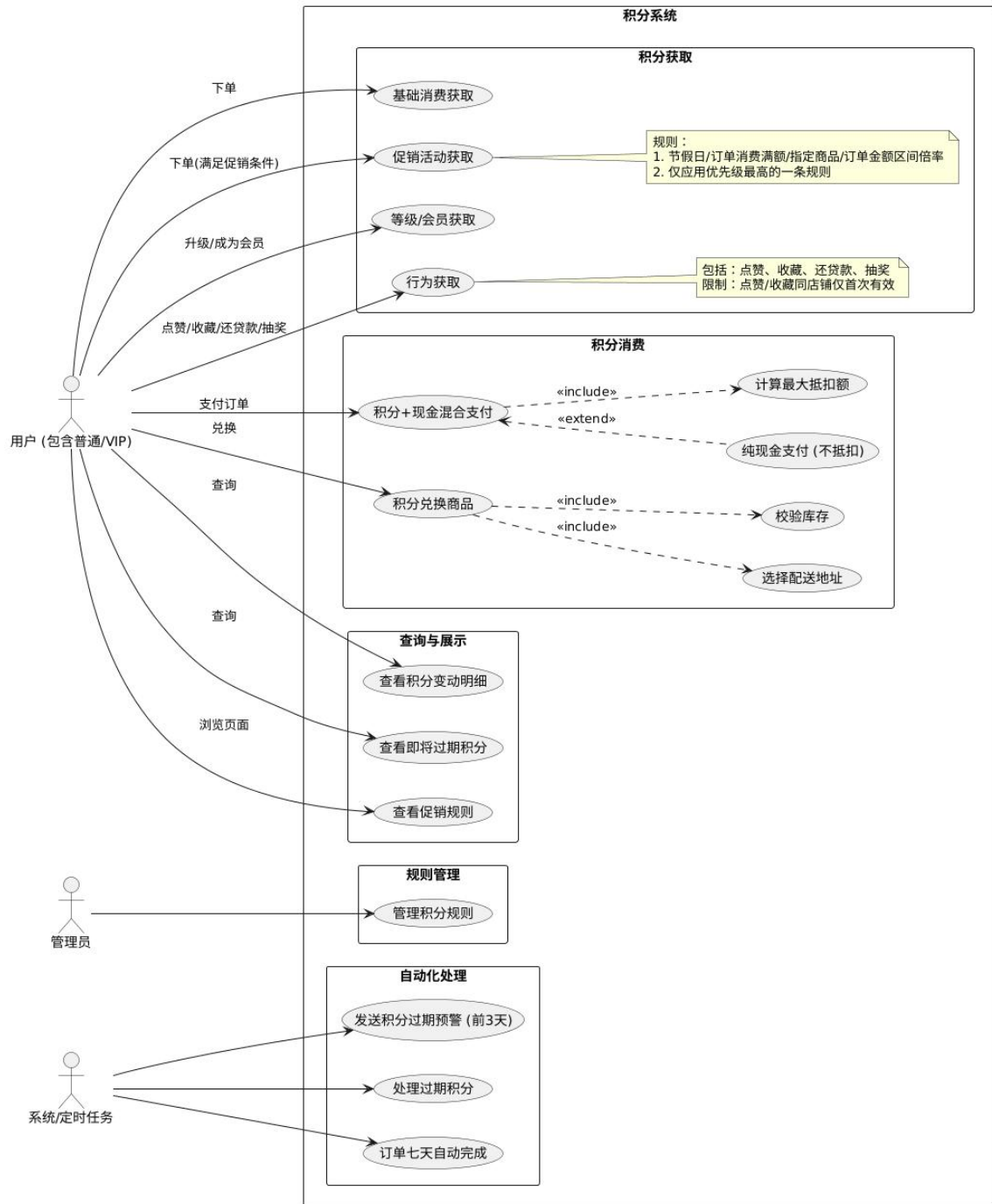


图 1-6 积分系统用例图

表 1-2 基础消费获取积分用例说明

编号	UC-001	名称	基础消费获取积分
执行者	用户	优先级	高
描述	用户通过下单消费，按消费金额和用户vip等级对应的比例获得基础积分。		
前置条件	用户账号有效，且订单支付成功。		
基本流程	1. 用户完成订单支付。 2. 系统根据订单实付金额和用户等级计算基础积分。 3. 系统将积分计入用户积分账户。		
结束状况	用户成功获得基础消费积分。		
可选流程	-		
异常流程	支付失败或订单取消，则不发放积分。		
备注	支付成功后积分计入用户账户时处于冻结状态，当订单完成后解冻积分；使用积分兑换商品时不获取基础消费积分。		

表 1-3 促销活动获取积分用例说明

编号	UC-002	名称	促销活动获取积分
执行者	用户	优先级	高
描述	用户在满足特定促销规则（如节假日、订单满额、指定商品、订单金额在某个区间等）时，获得额外倍数积分。		
前置条件	用户已完成订单支付，且订单满足至少一条有效的促销规则。		
基本流程	1. 用户完成订单支付。 2. 系统检索所有该订单适用的促销规则。 3. 系统根据预设优先级，选择最高的一条规则应用。 4. 系统基于基础积分和规则倍数，计算促销积分。 5. 系统将促销积分计入用户积分账户。		
结束状况	用户成功获得促销倍数积分。		
可选流程	-		
异常流程	-		
备注	促销规则包括：节假日倍率、订单消费满额倍率、指定商品倍率、订单金额区间倍率及其组合规则。若是指定商品的促销规则，可以从入口进入指定商品展示页面，选择购买后进入该商品的支付界面。		

表 1-4 成为/升级会员获取积分用例说明

编号	UC-003	名称	成为/升级会员获取积分
执行者	用户	优先级	中
描述	普通用户成为会员或会员升级时，获得对应的积分奖励。		
前置条件	用户成功支付相应会员费用。		
基本流程	1. 用户完成会员开通或升级操作。 2. 系统根据新的会员等级，确定对应的奖励积分数额。 3. 系统将奖励积分一次性计入用户积分账户。		
结束状况	用户成功获得会员等级积分奖励。		
可选流程	-		
异常流程	支付失败或升级条件未满足，则不发放积分。		
备注	不同等级的会员（白银、黄金、钻石），奖励的积分数不同。		

表 1-5 行为获取积分用例说明

编号	UC-004	名称	行为获取积分
执行者	用户	优先级	中
描述	用户通过完成特定行为（如点赞店铺、收藏店铺、还贷款、抽奖）获得积分奖励。		
前置条件	用户账号有效，且执行了指定行为。		
基本流程	1. 用户执行有效行为（如首次点赞某店铺）。 2. 系统校验该行为是否满足积分奖励条件。 3. 系统根据用户等级和行为类型，发放对应数额的积分。		
结束状况	用户成功获得行为积分。		
可选流程	-		
异常流程	如非首次点赞/收藏，或行为无效，则不发放积分。		
备注	为防止刷分，对同一店铺的点赞/收藏，只有第一次有效；不同vip等级的用户所获行为积分数、每月抽奖机会、对应奖池与中奖概率也不一样。		

表 1-6 积分+现金混合支付用例说明

编号	UC-005	名称	积分+现金混合支付
执行者	用户	优先级	高
描述	用户支付订单时，默认使用积分抵扣部分金额，剩余部分用现金支付。		
前置条件	用户有待支付订单，且账户中有可用积分。		
基本流程	1. 用户进入支付页面。 2. 系统自动计算并展示用户当前积分可抵扣的最大金额及优惠后需支付的现金。 3. 用户确认支付方式并完成支付。		
结束状况	订单支付成功，积分被扣除，现金完成支付。		
可选流程	-		
异常流程	-		
备注	支付完成后扣除的积分进入冻结状态，若用户完成订单则真正扣除；若用户取消订单则返还积分。		

表 1-7 积分兑换商品用例说明

编号	UC-006	名称	积分兑换商品
执行者	用户	优先级	高
描述	用户使用积分兑换指定商品。		
前置条件	用户拥有足额积分，且目标兑换商品有库存。		
基本流程	1. 用户浏览积分商城，选择可兑换商品。 2. 系统校验用户积分和商品库存。 3. 用户确认兑换并选择配送地址。 4. 系统扣除积分，锁定商品库存，生成不可取消的兑换订单。		
结束状况	商品兑换成功，积分被扣除。		
可选流程	-		
异常流程	积分不足或库存不足，兑换失败。		
备注	兑换订单不允许用户或商家取消，积分不予退回。不经过购物车流程。		

表 1-8 查看积分变动明细用例说明

编号	UC-007	名称	查看即将过期积分
执行者	用户	优先级	中
描述	用户查询自己账户的积分收入与支出详细记录。		
前置条件	用户已登录。		
基本流程	1. 用户进入“我的积分”页面。 2. 系统将记录（包括交易类型、来源、描述、数量、时间等）渲染并展示给用户。		
结束状况	用户成功查看到积分明细列表。		
可选流程	-		
异常流程	-		
备注	-		

表 1-9 查看即将过期积分用例说明

编号	UC-008	名称	查看即将过期积分
执行者	用户	优先级	中
描述	用户查询自己账户中即将在短期内过期的积分数额及具体过期时间。		
前置条件	用户已登录，且存在即将过期的积分。		
基本流程	1. 用户进入积分页面。 2. 页面显示有多少积分将在什么时候过期（即过期日期离现在最近的一条积分记录），点击可查看积分过期列表，按照过期日期离现在由近到远排列。		
结束状况	用户清晰知悉有多少积分即将过期。		
可选流程	-		
异常流程	-		
备注	-		

表 1-10 查看促销规则用例说明

编号	UC-009	名称	查看促销规则
执行者	用户	优先级	低
描述	用户在前端页面浏览当前所有有效的积分获取促销规则。		
前置条件	系统已配置有效的促销规则。		
基本流程	1. 用户进入促销活动页面或积分规则说明页面。 2. 系统从后台获取所有当前有效的促销规则。 3. 系统将规则以易于理解的方式渲染并展示给用户。		
结束状况	用户了解到当前可参与的积分促销活动。		
可选流程	-		
异常流程	-		
备注	-		

表 1-11 管理积分规则用例说明

编号	UC-010	名称	管理积分规则
执行者	管理员	优先级	高
描述	管理员在管理后端对积分系统的所有规则（包括基础比例和促销规则）进行创建、查询、修改和删除。		
前置条件	管理员已登录管理系统，并拥有相应权限。		
基本流程	1. 管理员进入积分规则管理模块。 2. 系统展示现有规则列表。 3. 管理员可执行新增、编辑、删除或查询详情等操作。 4. 系统保存更改并更新规则状态。		
结束状况	积分规则被成功更新。		
可选流程	-		
异常流程	-		
备注	-		

表 1-12 发送积分过期预警用例说明

编号	UC-011	名称	发送积分过期预警（前3天）
执行者	系统/定时任务	优先级	中
描述	系统在用户积分即将过期前3天，自动向用户的消息中心发送预警通知。		
前置条件	系统存在即将在3天后过期的积分记录。		
基本流程	1. 定时任务触发，扫描即将在3天后过期的积分记录。 2. 将预警消息发送给对应用户。		
结束状况	用户收到积分过期预警消息。		
可选流程	-		
异常流程	-		
备注	-		

表 1-13 处理过期积分用例说明

编号	UC-012	名称	处理过期积分
执行者	系统/定时任务	优先级	高
描述	系统自动扫描并作废已过有效期的用户积分。		
前置条件	系统存在已过有效期且未被使用的积分记录。		
基本流程	1. 定时任务（如每日凌晨）触发。 2. 系统扫描所有状态为“有效”但过期时间已早于当前时间的积分记录。 3. 系统将这些记录的状态更新为“已过期”，并记录一条积分扣除的流水。		
结束状况	过期积分被成功清理，用户积分总额相应减少。		
可选流程	-		
异常流程	-		
备注	-		

表 1-14 订单七天自动完成用例说明

编号	UC-013	名称	订单七天自动完成
执行者	系统/定时任务	优先级	中
描述	对于已支付且用户或商家未手动操作的订单，系统在七天后自动将其状态更新为“完成”。		
前置条件	订单处于“已支付”状态，且支付成功时间已超过七天。		
基本流程	1. 定时任务触发，扫描符合条件的订单。 2. 系统自动将订单状态更新为“完成”。 3. 执行订单完成后的后续逻辑（如结算等）。		
结束状况	订单状态自动变更为“完成”。		
可选流程	-		
异常流程	-		
备注	-		

第二章 模块设计

2.1 数据库设计

2.1.1 DB 一览表（新增部分）

表 2-1 新增数据库表一览

NO	数据库表名	说明
1	points_account	积分账户表
2	points_transaction	积分明细表
3	points_expiration	积分过期记录表
4	marketing_points_rule	营销系统-积分规则表
5	marketing_points_rule_food	营销系统-积分规则与商品关联表
6	marketing_points_exchange_rule	营销系统-积分兑换规则表
7	points_exchange_order	积分兑换订单表
8	points_expiration_alert_config	积分到期预警配置表
9	points_expiration_alert_log	积分到期预警记录表
10	points_lottery_rule	积分抽奖规则表
11	points_lottery_record	积分抽奖记录表

2.1.2 表结构

对于表约束，使用简写，说明如表2-2。

表 2-2 约束说明

标识	英文	解释
PK	primary key	主键
FK	foreign key	外键
NN	not null	非空
UQ	unique	唯一索引
AI	auto increment	自增长列
DEL	delete	逻辑删除标记 0-正常 1-删除

表 2-3 points_account 积分账户表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	user_id	bigint			NN UQ FK	用户ID（关联users表）
3	total_points	bigint		0	NN	总积分余额
4	available_points	bigint		0	NN	可用积分余额
5	frozen_points	bigint		0	NN	冻结积分（用于订单处理中）
6	member_level	tinyint		0	NN	会员等级 0-普通 1-白银 2-黄金 3-钻石
7	create_time	timestamp		CURRENT_TIMESTAMP		创建时间
8	update_time	timestamp		CURRENT_TIMESTAMP		更新时间
9	creator	bigint		NULL		创建人ID
10	updater	bigint		NULL		更新人ID
11	is_deleted	tinyint		0	DEL	是否删除

表 2-4 points_transaction 积分明细表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	user_id	bigint			NN FK	用户ID（关联users表）
3	account_id	bigint			NN FK	积分账户ID（关联points_account表）
4	transaction_type	tinyint			NN	交易类型 0-获得 1-消费 2-过期 3-冻结 4-解冻
5	points_source	tinyint			NN	积分来源 0-消费积分 1-促销积分 2-等级积分 3-行为积分 4-兑换商品 5-积分+现金消费

6	points_change	bigint		NN	积分变动数量（正数表示增加，负数表示减少）
7	points_balance	bigint		NN	变动后积分余额
8	expire_time	timestamp	NULL		积分过期时间
9	related_order_id	bigint	NULL		关联订单ID（关联orders表）
10	related_food_id	bigint	NULL		关联商品ID（关联food表，兑换商品时使用）
11	related_rule_id	bigint	NULL		关联积分规则ID（关联marketing_points_rule表）
12	description	varchar	500	NULL	交易描述
13	create_time	timestamp	CURRENT_TIMESTAMP		创建时间
14	creator	bigint	NULL		创建人ID
15	update_time	timestamp	CURRENT_TIMESTAMP		更新时间
16	updater	bigint	NULL		更新人ID
17	is_deleted	tinyint	0	DEL	是否删除

表 2-5 points_expiration 积分过期记录表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	user_id	bigint			NN FK	用户ID（关联users表）
3	transaction_id	bigint			NN FK	积分明细ID（关联points_transaction表）
4	points_amount	bigint			NN	即将过期积分数量
5	expire_time	timestamp			NN	过期时间（精确到秒）
6	expire_date	date			NN	过期日期（用于按日期查询）
7	is_expired	tinyint		0		是否已过期 0-未过期 1-已过期
8	create_time	timestamp		CURRENT_TIMESTAMP		创建时间

表 2-6 marketing_points_rule 营销系统-积分规则表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	rule_name	varchar	100		NN	规则名称
3	rule_type	tinyint			NN	规则类型 0-消费积分 1-促销积分 2-等级积分 3-行为积分
4	rule_status	tinyint		1	NN	规则状态 0-禁用 1-启用
5	points_ratio	decimal	10,4	NULL		积分比例（如0.1表示消费1元获得0.1积分）
6	points_multiplier	decimal	10,4	NULL		积分倍数（促销时使用，如2.0表示双倍积分）
7	member_level	tinyint		NULL		适用会员等级（NULL表示所有等级）
8	min_order_amount	decimal	10,2	NULL		最低订单金额（促销积分使用）
9	max_order_amount	decimal	10,2	NULL		最高订单金额（促销积分使用）
10	holiday_start	date		NULL		节假日开始日期
11	holiday_end	date		NULL		节假日结束日期
12	behavior_type	varchar	50	NULL		行为类型（like-点赞 collect-收藏 repay_loan-还贷款）
13	points_amount	bigint		NULL		固定积分数量（行为积分和等级积分使用）
14	expire_days	int		NULL		积分有效期（天数，NULL表示永久有效）
15	start_time	timestamp		NULL		规则生效开始时间
16	end_time	timestamp		NULL		规则生效结束时间
17	priority	int		0	NN	优先级（数字越大优先级越高）
18	create_time	timestamp		CURRENT_TIMESTAMP		创建时间
19	update_time	timestamp		CURRENT_TIMESTAMP		更新时间
20	creator	bigint		NULL		创建人ID
21	updater	bigint		NULL		更新人ID
22	is_deleted	tinyint		0	DEL	是否删除

表 2-7 marketing_points_rule_food 营销系统-积分规则与商品关联表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	rule_id	bigint			NN FK	积分规则ID（关联marketing_points_rule表）
3	food_id	bigint			NN FK	商品ID（关联food表）
4	create_time	timestamp		CURRENT_T IMESTAMP		创建时间
5	update_time	timestamp		CURRENT_T IMESTAMP		更新时间
6	is_deleted	tinyint		0	NN DEL	是否删除

表 2-8 marketing_points_exchange_rule 营销系统-积分兑换规则表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	rule_name	varchar	100		NN	规则名称
3	rule_type	tinyint			NN	规则类型 0-积分+现金 1-兑换商品
4	rule_status	tinyint		1	NN	规则状态 0-禁用 1-启用
5	exchange_ratio	decimal	10,4	NULL		兑换比例（如10表示10积分=1元）
6	min_points	bigint		NULL		最小使用积分数量
7	max_points	bigint		NULL		最大使用积分数量（NULL表示不限制）
8	food_id	bigint		NULL	FK	商品ID（关联food表，兑换商品时使用）
9	required_points	bigint		NULL		所需积分（兑换商品时使用）
10	stock_quantity	int		NULL		库存数量（兑换商品时使用）
11	start_time	timestamp		NULL		规则生效开始时间
12	end_time	timestamp		NULL		规则生效结束时间
13	create_time	timestamp		CURRENT_T IMESTAMP		创建时间
14	update_time	timestamp		CURRENT_T IMESTAMP		更新时间
15	creator	bigint		NULL		创建人ID
16	updater	bigint		NULL		更新人ID
17	is_deleted	tinyint		0	DEL	是否删除

表 2-9 points_exchange_order 积分兑换订单表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	user_id	bigint			NN FK	用户ID（关联users表）
3	order_id	bigint		NULL	FK	关联订单ID（关联orders表）
4	food_id	bigint		NULL	FK	兑换商品ID（关联food表）
5	points_used	bigint			NN	使用积分数量
6	cash_amount	decimal	10,2	NULL		现金金额（预留字段，当前纯积分兑换为0）
7	status	tinyint		0	NN	状态 0-待处理 1-已完成 2-已取消
8	create_time	timestamp		CURRENT_TIMESTAMP		创建时间
9	update_time	timestamp		CURRENT_TIMESTAMP		更新时间
10	is_deleted	tinyint		0	DEL	是否删除

表 2-10 points_expiration_alert_config 积分到期预警配置表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	alert_days	int			NN	提前预警天数（如7表示提前7天预警）
3	alert_cycle	int		NULL		预警周期（天数，如每天、每3天）
4	sms_template	varchar	500	NULL		短信模板（支持变量）
5	is_enabled	tinyint		1		是否启用 0-禁用 1-启用
6	create_time	timestamp		CURRENT_TIMESTAMP		创建时间
7	update_time	timestamp		CURRENT_TIMESTAMP		更新时间

表 2-11 points_expiration_alert_log 积分到期预警记录表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	user_id	bigint			NN FK	用户ID（关联users表）
3	points_amount	bigint			NN	即将过期积分数量
4	expire_date	date			NN	过期日期
5	alert_time	timestamp			NN	预警时间
6	next_alert_time	timestamp		NULL		下次预警时间（根据预警周期计算）
7	is_sent	tinyint		0		是否已发送 0-未发送 1-已发送
8	phone	varchar	20	NULL		用户手机号（发送短信时使用）
9	create_time	timestamp		CURRENT_TIMESTAMP		创建时间

表 2-12 points_lottery_rule 积分抽奖规则表

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	rule_name	varchar	100		NN	规则名称
3	member_level	tinyint			NN	适用会员等级 1-白银 2-黄金 3-钻石
4	prize_type	tinyint			NN	奖品类型 0-没中奖 1-固定积分 2-积分翻倍
5	prize_points	bigint		NULL		固定积分数量（仅 prize_type=1时有效）
6	prize_multiplier	decimal	10,2	NULL		积分翻倍倍数（仅 prize_type=2时有效）
7	probability	int			NN	中奖概率（百分比，0-100）
8	prize_order	int		0		奖品排序（数字越小越靠前）
9	prize_description	varchar	200	NULL		奖品描述（用于前端展示）
10	rule_status	tinyint		1		规则状态 0-禁用 1-启用
11	start_time	timestamp		NULL		规则生效开始时间
12	end_time	timestamp		NULL		规则生效结束时间
13	create_time	timestamp		CURRENT_TIMESTAMP		创建时间
14	update_time	timestamp		CURRENT_TIMESTAMP		更新时间

15	creator	bigint	NULL	创建人ID
16	updater	bigint	NULL	更新人ID
17	is_deleted	tinyint	0	是否删除

表 2-13 points_lottery_record 积分抽奖表记录

NO	Field	Type	Size	Default	Constraint	Comment
1	id	bigint			NN PK AI	主键ID
2	user_id	bigint			NN FK	用户ID（关联users表）
3	member_level	tinyint			NN	抽奖时的会员等级 0-普通 1-白银 2-黄金 3-钻石
4	lottery_type	tinyint			NN	抽奖类型 0-没中奖 1-固定积分 2-积分翻倍
5	points_reward	bigint		NULL		获得的积分数量（没中奖时为0，积分翻倍时为NULL）
6	points_multiplier	decimal	10,2	NULL		积分翻倍倍数（仅积分翻倍时使用）
7	original_points	bigint		NULL		翻倍前的积分数量（仅积分翻倍时使用）
8	lottery_month	varchar	7		NN	抽奖月份（格式：YYYY-MM）
9	transaction_id	bigint		NULL	FK	关联积分明细ID（关联points_transaction表）
10	create_time	timestamp		CURRENT_T IMESTAMP		创建时间
11	update_time	timestamp		CURRENT_T IMESTAMP		更新时间
12	creator	bigint		NULL		创建人ID
13	updater	bigint		NULL		更新人ID
14	is_deleted	tinyint		0		是否删除

表 2-14 orders 表新增字段

NO	Field	Type	Size	Default	Constraint	Comment
1	points_used	bigint		0		使用积分数量
2	points_amount	bigint		0		获得积分数量
3	points_discount _amount	decimal	10,2	0		积分抵扣的现金金额（元）

2.2 后端接口设计

为方便响应处理与异常情况的判断，统一将接口响应的数据模型统一为 `HttpResult`，具体说明如表2-8。

表 2-15 `HttpResult` 包装类字段说明

字段名	类型	说明
<code>success</code>	<code>boolean</code>	是否出现异常
<code>code</code>	<code>string</code>	异常状态码
<code>data</code>	<code>object</code>	返回数据
<code>message</code>	<code>string</code>	异常信息

2.2.1 新增接口

(1) 营销系统-积分管理接口

1. 查询积分规则列表

接口路径: `/api/marketing/points/rules`

请求方式: GET

参数:

- `ruleType` (选填): 规则类型
- `pageNum` (选填): 页码
- `pageSize` (选填): 每页大小

返回值: `HttpResult<List<PointsRuleVO>>`

2. 创建积分规则

接口路径: `/api/marketing/points/rules`

请求方式: POST

参数:

- `body` (必填): 积分规则数据

返回值: `HttpResult<Long>`

3. 更新积分规则

接口路径: `/api/marketing/points/rules/{id}`

请求方式: PUT

参数:

- id (必填): 规则ID
- body (必填): 更新的规则数据

返回值: `HttpResult<Boolean>`

4. 删除积分规则

接口路径: `/api/marketing/points/rules/{id}`

请求方式: DELETE

参数:

- id (必填): 规则ID
- 返回值: `HttpResult<Boolean>`

5. 查询积分兑换规则列表

接口路径: `/api/marketing/points/exchange-rules`

请求方式: GET

参数:

- ruleType (选填): 规则类型
- pageNum (选填): 页码
- pageSize (选填): 每页大小

返回值: `HttpResult<List<PointsExchangeRuleVO>>`

6. 创建积分兑换规则

接口路径: `/api/marketing/points/exchange-rules`

请求方式: POST

参数:

- body (必填): 兑换规则数据

返回值: `HttpResult<Long>`

7. 更新积分兑换规则

接口路径: /api/marketing/points/exchange-rules/{id}

请求方式: PUT

参数:

- id (必填): 规则ID
- body (必填): 更新的规则数据

返回值: `HttpResult<Boolean>`

8. 删除积分兑换规则

接口路径: /api/marketing/points/exchange-rules/{id}

请求方式: DELETE

参数:

- id (必填): 规则ID

返回值: `HttpResult<Boolean>`

9. 查询预警配置

接口路径: /api/marketing/points/alert-config

请求方式: GET

参数: 无

返回值: `HttpResult<PointsExpirationAlertConfig>`

10. 更新预警配置

接口路径: /api/marketing/points/alert-config

请求方式: PUT

参数:

- body (必填): 预警配置数据

返回值: `HttpResult<Boolean>`

11. 创建积分抽奖规则

接口路径: /api/marketing/points/lottery-rules

请求方式: POST

参数:

- body (必填): 积分抽奖规则信息

返回值: `HttpResult<Long>`

12. 更新积分抽奖规则

接口路径: `/api/marketing/points/lottery-rules/{id}`

请求方式: PUT

参数:

- id (必填): 规则ID
- body (必填): 积分抽奖规则更新数据

返回值: `HttpResult<Boolean>`

13. 删除积分抽奖规则

接口路径: `/api/marketing/points/lottery-rules/{id}`

请求方式: DELETE

参数:

- id (Long, 必填): 规则ID

返回值: `HttpResult<Boolean>`

14. 查询积分抽奖规则列表

接口路径: `/api/marketing/points/lottery-rules`

请求方式: GET

参数:

- memberLevel (Integer, 可选): 会员等级筛选 1-白银 2-黄金 3-钻石
- ruleStatus (Integer, 可选): 规则状态筛选 0-禁用 1-启用
- pageNum (Integer, 可选, 默认值: 1): 页码
- pageSize (Integer, 可选, 默认值: 10): 每页数量

返回值: `HttpResult<List<PointsLotteryRuleVO>>`

(2) 积分系统-用户接口

1. 查询积分账户

接口路径: /api/points/account

请求方式: GET

参数: 无

返回值: `HttpResult<PointsAccountVO>`

2. 查询积分明细

接口路径: /api/points/transactions

请求方式: GET

参数:

- pageNum (选填): 页码
- pageSize (选填): 每页大小
- transactionType (选填): 交易类型
- pointsSource (选填): 积分来源

返回值: `HttpResult<List<PointsTransactionVO>>`

3. 积分兑换商品

接口路径: /api/points/exchange

请求方式: POST

参数:

- body (必填): 兑换信息

返回值: `HttpResult<Long>`

4. 查询即将过期的积分记录

接口路径: /api/points/expiring

请求方式: GET

参数: 无

返回值: `HttpResult<List<PointsExpirationVO>>`

5. 统计即将过期的积分总数

接口路径: /api/points/expiring/count

请求方式: GET

参数: 无

返回值: `HttpResult<Long>`

6. 查询已过期的积分记录

接口路径: /api/points/expired

请求方式: GET

参数: 无

返回值: `HttpResult<List<PointsExpirationVO>>`

7. 获取积分+现金兑换比例

接口路径: /api/points/exchange-ratio

请求方式: GET

参数: 无

返回值: `HttpResult<BigDecimal>`

8. 获取可兑换商品列表

接口路径: /api/points/exchange-goods

请求方式: GET

参数: 无

返回值: `HttpResult<List<PointsExchangeGoodsVO>>`

9. 计算可抵扣金额

接口路径: /api/points/deductible-amount

请求方式: GET

参数:

- `orderAmount` (必填): 订单金额

返回值: `HttpResult<BigDecimal>`

10. 执行抽奖

接口路径: `/api/points/lottery/draw`

请求方式: `POST`

参数: 无

返回值: `HttpResult<PointsLotteryResultVO>`

11. 查询抽奖记录

接口路径: `/api/points/lottery/records`

请求方式: `GET`

参数:

- `limit: integer` (可选) - 限制返回记录数

返回值: `HttpResult<List<PointsLotteryRecordVO>>`

12. 获取抽奖信息

接口路径: `/api/points/lottery/info`

请求方式: `GET`

参数: 无

返回值: `HttpResult<PointsLotteryInfoVO>`

2.3 前端设计

2.3.1 用户端

1.积分系统入口

入口位于“我的”页面中，如图2-1。

2.用户积分管理 (图2-2)

- ① 积分总额: 展示醒目的当前积分数量
- ② 抽奖入口: 可进入积分抽奖页面
- ③ 即将过期积分入口: 可进入即将过期积分展示页面
- ④ 可兑换商品列表: 显示库存等基础信息, 点击兑换跳转至地址选择页面



图 2-1 积分系统入口

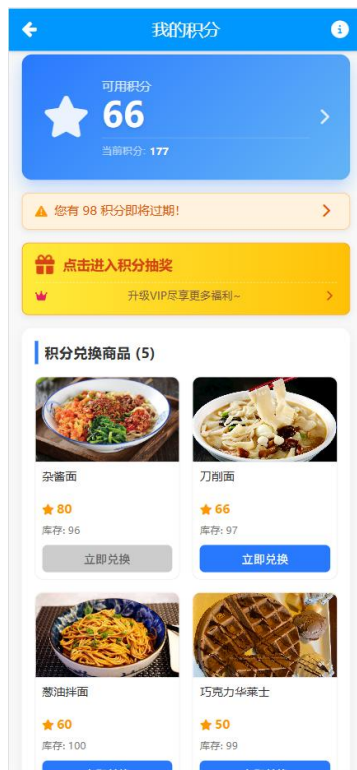


图 2-2 用户积分管理页面



图 2-3 积分规则弹窗

3.积分明细 (图2-4, 2-5)

- ① 积分展示: 展示当前积分数量与可用的积分数量
- ② 条件筛选: 可筛选查看不同来处与去向的积分变动明细

4.即将过期积分 (图2-6)

- ① 即将过期积分总额: 展示醒目的即将过期积分数量

② 即将过期积分列表：展示剩余过期天数、日期与积分数量



图 2-4 积分明细页面1



图 2-5 积分明细页面2



图 2-6 即将过期积分页面

5.积分抽奖 (图2-7, 2-8)

① 活动规则：点击将展示规则信息弹窗

② 抽奖机会：与虚拟钱包VIP规则关联，展示当前会员信息，点击获得更多抽奖机会将跳转至虚拟钱包会员申请页面

③ 积分抽奖转盘：根据当前用户剩余抽奖次数控制是否可以抽奖，点击中间抽奖方块可进行抽奖

④ 抽奖结果记录：展示已有抽奖结果



图 2-7 抽奖规则



图 2-8 积分抽奖页面

6.首页促销积分展示 (图2-9)

首页添加积分公告轮播图，展示一些积分相关的公告，特定时间存在的积分优惠规则等。同时例如对于购买指定商品的积分优惠可直接跳转至促销商品页面。

7.促销商品页面 (图2-10)

通过首页跳转按钮前往查看促销指定商品，购买指定商品可以获得双倍积分，点击立即购买后即为正常下单流程。



图 2-9 积分公告轮播图

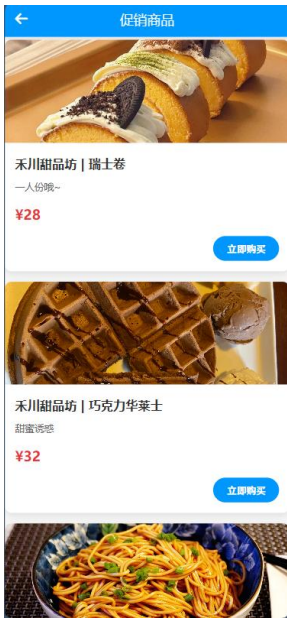


图 2-10 积分优惠商品页面

8.积分+现金支付 (图2-11)

支付页面优化：支付时候自动勾选积分抵扣，也可以取消；成功支付页面显示支付明细（积分抵扣金额）

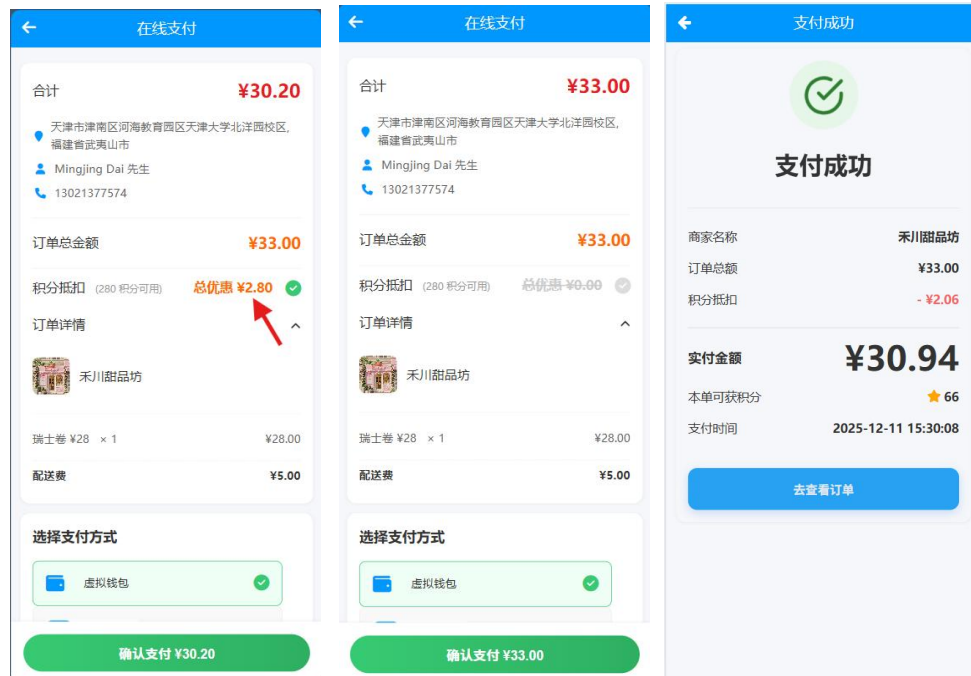


图 2-11 更新后的支付页面

9.积分预警

根据数据库里配置的预警规则，在积分过期的三天前会向对应用户的消息与通知发送通知提醒。

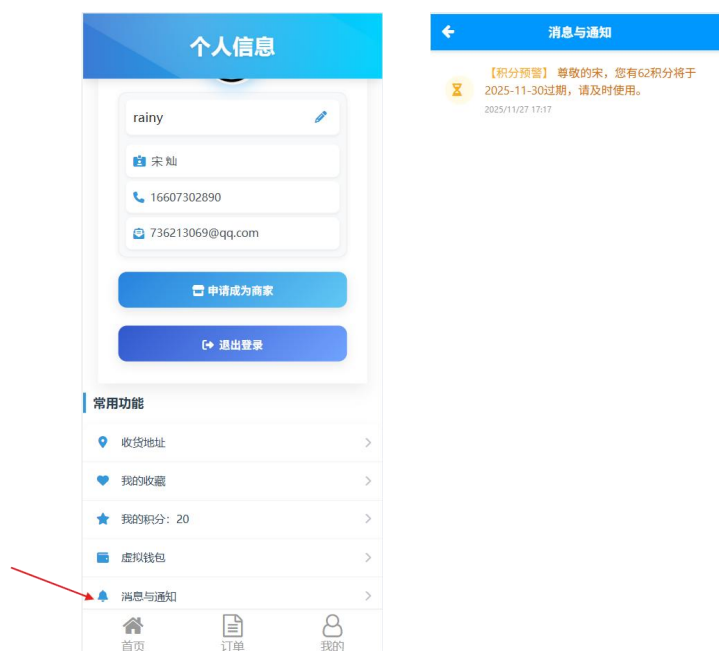


图 2-12 积分预警页面

2.3.2 管理端

1.营销中心入口(图2-13)

- ① 积分促销规则: 查看当前生效的积分规则详情 (图2-14)
- ② 规则管理: 点击跳转至规则管理页面, 用于对积分相关规则的增删改
- ③ 预警配置: 点击跳转至预警配置界面, 用于积分预警消息的管理
- ④ 兑换管理: 点击跳转至兑换管理, 用于管理积分兑换相关内容



图 2-13 营销中心



图 2-14 生效积分规则弹窗

2.规则管理

- ① 搜索规则: 可通过搜索框搜索相关规则
- ② 条件筛选: 支持筛选规则的类型、状态、创建时间
- ③ 规则列表: 展示筛选出的规则内容, 点击item弹出详细信息弹窗, 进行增删改操作



图 2-15 积分使用规则管理页面



图 2-16 新增使用规则弹窗

3.预警配置 (图2-17)

- ① 启用/禁用预警：选择是否开启积分预警提醒用户
- ② 预警基础配置：设置提前预警天数、周期等
- ③ 模板设置：编辑预警消息的内容模板

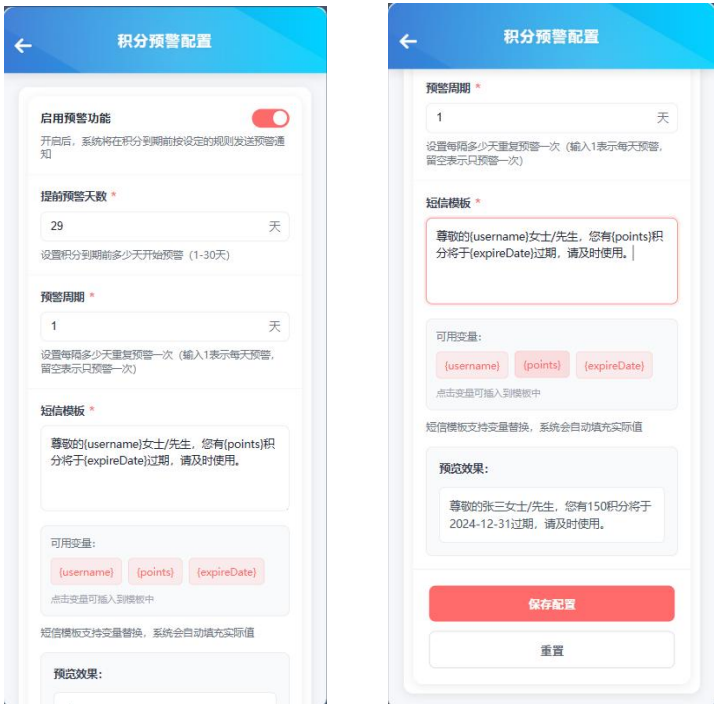


图 2-17 积分预警配置

4.兑换管理 (图2-18)

- ① 积分-现金管理: 可筛选指定状态的规则, 并编辑、禁用、新增规则
- ② 兑换商品管理: 可筛选指定状态的规则, 并编辑、禁用、新增可兑换商品

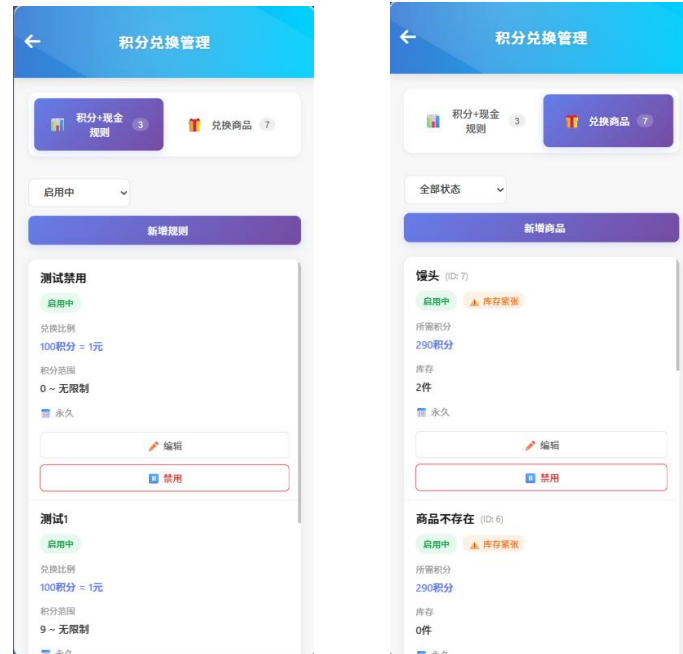


图 2-18 积分兑换管理

第三章 模块实现

3.1 整体架构设计

积分系统采用分层架构，严格遵循面向对象设计原则。系统分为 Controller、Service、Mapper 三层，每层职责明确。Controller 层只处理 HTTP 请求和响应，不包含业务逻辑，体现单一职责原则。Service 层实现业务逻辑，通过接口定义契约，实现类实现接口，体现依赖倒置原则和接口隔离原则。Mapper 层负责数据访问，使用 MyBatis 实现，体现数据访问层与业务逻辑层的分离。

积分系统采用分层架构，包含积分系统和营销系统，关联订单系统、商家互动（点赞/收藏）模块、虚拟钱包模块。积分系统负责账户操作（增加、减少、查询），营销系统负责规则管理和积分计算。系统间通过同步API调用和异步消息队列交互：营销系统与积分系统使用同步调用，订单系统在订单支付成功后使用 RabbitMQ 异步通信通知营销系统，这种设计使系统易于扩展和维护，体现了开闭原则。其余业务逻辑下使用同步调用。

依赖注入贯穿整个系统。所有 Service 实现类使用 @Service 注解，由 Spring 容器管理生命周期。依赖关系通过 @Autowired 注入，而不是在类内部创建，体现控制反转原则。核心服务类使用 @Transactional 保证数据一致性，关键操作使用数据库行锁（SELECT ... FOR UPDATE）防止并发问题。

另外，项目还集成了Redis，采用Cache-Aside（旁路缓存）模式，针对用户积分账户、积分兑换比例和即将过期的积分列表等高频查询数据进行缓存，并设置了合理的过期时间。为确保数据强一致性，在所有积分变动（如增加、扣减、冻结）操作后，会主动删除相关的缓存数据，从而保证下一次读取时能从数据库加载最新结果，有效避免了数据不一致的问题。

3.2 业务逻辑实现

3.2.1 积分系统核心模块的详细实现

1.增加积分的实现机制

addPoints 方法使用 @Transactional 保证原子性。首先通过 selectForUpdate 加行锁查询或创建积分账户，避免并发创建。为了进一步完善业务逻辑，在新用户注册时会同时创建该用户的积分账户。

若增加积分的来源是用户支付订单的奖励积分，需要将该积分冻结，直到订单完成或取消。在 addPoints 中判断是否为订单奖励积分：pointsSource == 0 且 relatedOrderId != null 时，增加总积分和冻结积分，不增加可用积分；其他来源积分直接增加可用积分。

```

// 判断是否是订单奖励积分（需要冻结）
boolean isOrderReward = (pointsAddDTO.getPointsSource() == 0 &&
    pointsAddDTO.getRelatedOrderId() != null);

Long newTotalPoints = account.getTotalPoints() + pointsAddDTO.getPoints();
Long newAvailablePoints = account.getAvailablePoints();
Long newFrozenPoints = account.getFrozenPoints();

if (isOrderReward) {
    // 订单奖励积分：增加总积分和冻结积分，不增加可用积分
    newFrozenPoints = account.getFrozenPoints() + pointsAddDTO.getPoints();
} else {
    // 其他积分：直接增加可用积分
    newAvailablePoints = account.getAvailablePoints() + pointsAddDTO.getPoints();
}

```

然后调用 `validateAndSetAccountValues` 验证数据完整性：检查总积分、可用积分、冻结积分是否小于 0，验证总积分是否等于可用积分加冻结积分。验证失败抛出异常，不自动修正，遵循 Fail-Fast 原则。验证通过后更新积分账户，插入积分明细记录，创建过期记录，最后删除相关缓存。

2.扣减积分的优先过期算法实现

`deductPoints` 方法实现优先扣减即将过期积分的算法。首先使用 `selectForUpdate` 加行锁查询积分账户，检查可用积分是否充足。然后查询用户即将过期的积分记录，使用 `selectExpiringPoints` 方法，SQL 查询条件为 `is_expired = 0 AND expire_time > #{currentTime}` ORDER BY `expire_time` ASC，按过期时间升序返回。

```

// 优先扣减即将过期的积分
Long remainingPoints = pointsDeductDTO.getPoints();
LocalDateTime currentTime = LocalDateTime.now();
List<PointsExpiration> expiringPoints = pointsExpirationMapper.selectExpiringPoints(
    pointsDeductDTO.getUserId(), currentTime);

// 按过期时间升序扣减
for (PointsExpiration exp : expiringPoints) {
    if (remainingPoints <= 0) {
        break;
    }
}

```

```

Long deductAmount = Math.min(remainingPoints, exp.getPointsAmount());
exp.setPointsAmount(exp.getPointsAmount() - deductAmount);

if (exp.getPointsAmount() <= 0) {
    exp.setIsExpired(true);
}

pointsExpirationMapper.updateById(exp);

remainingPoints -= deductAmount;
}

```

算法遍历过期记录，每次扣减 `Math.min(remainingPoints, exp.getPointsAmount())`，确保不超过记录数量和剩余需扣减数量。扣减后更新过期记录，如果记录积分数量为0，标记为已过期。如果遍历完成后 `remainingPoints > 0`，说明即将过期的积分不足，抛出异常。最后更新积分账户余额，插入消费明细，删除缓存。

3.积分冻结与解冻的实现机制

积分冻结与解冻用于两种场景：订单奖励积分和积分+现金支付使用的积分的冻结与解冻。

在第一种场景下，订单支付完成后，通过 RabbitMQ 异步消息触发营销系统计算积分，调用 `PointsServiceImpl` 中的 `addPoints` 方法发放奖励积分。在 `addPoints` 中该奖励积分被判定为订单奖励积分，在获得时即被冻结：增加总积分和冻结积分，不增加可用积分。插入的交易记录为：`transaction_type = 0`（获得），`points_source = 0`（消费积分），描述为“订单消费获得积分（冻结中，订单完成后解冻）”。

订单完成时，调用 `unfreezeRewardPoints` 解冻奖励积分。该方法查询订单的奖励积分记录（`points_source = 0`，`related_order_id = orderId`），计算需解冻的积分总数，将冻结积分转为可用积分（总积分不变），插入解冻交易记录（`transaction_type = 4`，`points_source = 0`）。

```

@Override
@Transactional
public Boolean unfreezeRewardPoints(Long userId, Long orderId) {
    // 查询该订单的奖励积分记录（points_source = 0）

    List<PointsTransaction> rewardTransactions =
        pointsTransactionMapper.selectByOrderIdAndSourceList(

            orderId, 0); // 0-消费积分

```

```

// 计算需要解冻的积分总数
Long totalRewardPoints = 0L;
for (PointsTransaction trans : rewardTransactions) {
    if (Objects.equals(trans.getUserId(), userId) &&
        Objects.equals(trans.getRelatedOrderId(), orderId)) {
        totalRewardPoints += trans.getPointsChange();
    }
}

// 更新积分账户（解冻积分：从冻结积分转为可用积分）
Long newAvailablePoints = account.getAvailablePoints() + totalRewardPoints;
Long newFrozenPoints = account.getFrozenPoints() - totalRewardPoints;
// 总积分不变
validateAndSetAccountValues(account, account.getTotalPoints(), newAvailablePoints,
newFrozenPoints);

// 插入解冻交易记录（transaction_type = 4, points_source = 0）
// ...
}

```

订单取消时，调用 `cancelRewardPoints` 取消奖励积分。该方法查询订单的奖励积分记录，减少总积分和冻结积分（可用积分不变），逻辑删除积分明细和过期记录。

```

@Override
@Transactional
public Boolean cancelRewardPoints(Long userId, Long orderId) {
    // 1. 查询积分账户（带行锁）
    PointsAccount account = pointsAccountMapper.selectForUpdate(userId);
    if (account == null) {
        return false;
    }
    // 2. 查询该订单的奖励积分记录
    List<PointsTransaction> rewardTransactions =
pointsTransactionMapper.selectByOrderIdAndSourceList(
    orderId, 0); // 0-消费积分
    // 3. 计算需要取消的积分总数
    Long totalRewardPoints = 0L;

    for (PointsTransaction trans : rewardTransactions) {
        // 确保是当前用户的奖励积分记录
    }
}

```

```
        if (Objects.equals(trans.getUserId(), userId) &&
            Objects.equals(trans.getRelatedOrderId(), orderId)) {
            totalRewardPoints += trans.getPointsChange();
        }
    }

    // 4. 如果没有奖励积分记录，直接返回
    if (totalRewardPoints == 0) {
        return true;
    }

    // 5. 更新积分账户（减少总积分和冻结积分）
    Long newTotalPoints = account.getTotalPoints() - totalRewardPoints;
    Long newFrozenPoints = account.getFrozenPoints() - totalRewardPoints;
    // 验证并设置积分账户的值，确保不小于0
    validateAndSetAccountValues(account, newTotalPoints, account.getAvailablePoints(),
        newFrozenPoints);
    account.setUpdateTime(LocalDateTime.now());
    Long currentUserId = getCurrentUserId();
    account.setUpdater(currentUserId);
    pointsAccountMapper.updateById(account);

    // 6. 标记相关积分明细和过期记录为已删除
    for (PointsTransaction trans : rewardTransactions) {
        if (Objects.equals(trans.getUserId(), userId) &&
            Objects.equals(trans.getRelatedOrderId(), orderId)) {
            // 标记积分明细为已删除
            pointsTransactionMapper.deleteById(trans.getId(), LocalDateTime.now(),
                currentUserId);

            // 标记过期记录为已删除
            pointsExpirationMapper.deleteByTransactionId(trans.getId());
        }
    }

    // 删除相关缓存（写操作后清除缓存，保证数据一致性）
    pointsCacheService.deleteUserAllCache(userId);

    return true;
}
```

在第二种场景下，订单支付时，如果用户选择使用积分+现金支付（默认启用），`OrderServiceImpl.setOrderState` 方法会调用 `pointsService.calculateDeductibleAmount` 计算可抵扣金额，如果可抵扣金额大于 0，则调用 `freezePoints` 冻结积分。

```
// 计算可抵扣金额
BigDecimal deductibleAmount = pointsService.calculateDeductibleAmount(
    order.getCustomerId(), order.getOrderTotal());

if (deductibleAmount.compareTo(BigDecimal.ZERO) > 0) {
    // 获取积分兑换比例
    BigDecimal exchangeRatio = exchangeRuleService.getCashExchangeRatio();
    // 计算需要冻结的积分数量
    Long pointsToFreeze = deductibleAmount.multiply(exchangeRatio).longValue();

    // 冻结积分（优先扣减即将过期的积分）
    pointsService.freezePoints(order.getCustomerId(), pointsToFreeze, orderId);
}
```

`freezePoints` 方法将可用积分转为冻结积分（总积分不变）。使用 `selectForUpdate` 加行锁查询积分账户，检查可用积分是否充足，计算新的可用积分和冻结积分，验证数据完整性后更新账户，插入冻结交易记录（`transaction_type = 3`，`points_source = 5`，`points_change` 为负数），最后删除缓存。

```
@Override
@Transactional
public Boolean freezePoints(Long userId, Long points, Long orderId) {
    PointsAccount account = pointsAccountMapper.selectForUpdate(userId);
    if (account == null || account.getAvailablePoints() < points) {
        throw new APIException("POINTS_INSUFFICIENT", "积分不足");
    }

    // 将可用积分转为冻结积分（总积分不变）
    Long newAvailablePoints = account.getAvailablePoints() - points;
    Long newFrozenPoints = account.getFrozenPoints() + points;
    validateAndSetAccountValues(account, account.getTotalPoints(), newAvailablePoints,
        newFrozenPoints);

    // 插入冻结交易记录

    PointsTransaction transaction = new PointsTransaction();
```

```

transaction.setTransactionType(3); // 3-冻结
transaction.setPointsSource(5); // 5-积分+现金消费
transaction.setPointsChange(-points); // 负数表示减少
transaction.setRelatedOrderId(orderId);
transaction.setDescription("订单冻结积分");
// ...
}

```

订单取消时，调用 `unfreezePoints` 解冻积分。该方法查询订单的冻结交易记录（`transaction_type = 3`），计算需解冻的积分总数，将冻结积分转回可用积分（总积分不变），插入解冻交易记录（`transaction_type = 4`，`points_source = 5`）。

```

@Override
@Transactional
public Boolean unfreezePoints(Long userId, Long orderId) {
    // 查询该订单的冻结交易记录（transaction_type = 3）
    List<PointsTransaction> freezeTransactions =
        pointsTransactionMapper.selectByOrderIdAndType(
            orderId, 3); // 3-冻结

    // 计算需要解冻的积分总数
    Long totalFrozen = 0L;
    for (PointsTransaction trans : freezeTransactions) {
        if (Objects.equals(trans.getUserId(), userId)) {
            totalFrozen += Math.abs(trans.getPointsChange());
        }
    }

    // 更新积分账户（解冻积分：从冻结积分转回可用积分）
    Long newAvailablePoints = account.getAvailablePoints() + totalFrozen;
    Long newFrozenPoints = account.getFrozenPoints() - totalFrozen;
    // 总积分不变
    validateAndSetAccountValues(account, account.getTotalPoints(), newAvailablePoints,
        newFrozenPoints);

    // 插入解冻交易记录（transaction_type = 4, points_source = 5）
    // ...
}

```

订单完成时，调用 `deductFrozenPoints` 真正扣除已冻结的积分。该方法查询订单的冻结积分记录（`transaction_type = 3`，`points_source = 5`），计算总冻结积分，减少总积分和冻结积分（可用积分不变），同时按过期时间优先扣减即将过期的记录，最后插入消费明细（`transaction_type = 1`，`points_source = 5`）。

```
@Override
@Transactional
public Boolean deductFrozenPoints(Long userId, Long orderId) {
    // 查询该订单的冻结积分记录（transaction_type = 3）
    List<PointsTransaction> freezeTransactions =
        pointsTransactionMapper.selectByOrderIdAndType(
            orderId, 3); // 3-冻结

    // 过滤出积分+现金消费的冻结记录（points_source = 5）
    Long totalFrozen = 0L;
    for (PointsTransaction trans : freezeTransactions) {
        if (Objects.equals(trans.getPointsSource(), 5) &&
            Objects.equals(trans.getUserId(), userId) &&
            Objects.equals(trans.getRelatedOrderId(), orderId)) {
            totalFrozen += Math.abs(trans.getPointsChange()); // 冻结时是负数，取绝对值
        }
    }

    // 真正扣除积分：从冻结积分转为已扣除（减少总积分和冻结积分）
    Long newFrozenPoints = account.getFrozenPoints() - totalFrozen;
    Long newTotalPoints = account.getTotalPoints() - totalFrozen;
    // 可用积分不变
    validateAndSetAccountValues(account, newTotalPoints, account.getAvailablePoints(),
        newFrozenPoints);

    // 优先扣减即将过期的积分（更新过期记录）
    // ...

    // 插入消费明细（transaction_type = 1，points_source = 5）
    // ...
}
```

4.积分兑换商品的实现机制

`exchangeGoods` 方法实现积分兑换商品的完整流程。首先查询兑换规则，检查商品是否存在、规则是否启用、库存是否充足。然后先减少库存，使用 `WHERE stock_quantity >= #{quantity}` 条件保证库存充足，如果更新行数为 0，说明库存不足，抛出异常。

验证配送地址是否属于当前用户，然后创建普通订单（供商家查看和处理）。订单状态为 1（已支付），订单金额为 0（积分兑换），创建订单明细，调用 `pointsService.deductPoints` 扣除积分（优先扣减即将过期的积分），创建积分兑换订单记录，积分兑换商品订单不允许顾客或商家取消。

```
//订单状态为已取消
if (orderState == 4) {
    // 检查是否是积分兑换订单，积分兑换订单不允许取消
    if (isPointsExchangeOrder(orderId)) {
        throw new APIException("POINTS_EXCHANGE_ORDER_CANNOT_CANCEL", "
积分兑换订单不允许取消");
    }
    //...
}
```

整个流程使用 `@Transactional` 保证原子性：如果任何步骤失败，所有操作回滚，包括库存恢复、订单删除、积分恢复，保证数据一致性。

5.积分抽奖功能的实现机制

积分抽奖功能作为行为积分的一种特殊形式，允许会员用户通过每月限制次数的抽奖活动获取积分奖励。系统根据会员等级配置不同的抽奖次数限制、奖池规则和中奖概率，支持固定积分奖励和积分翻倍两种奖品类型。

统计本月已抽奖次数时，使用 `lottery_month` 字段（格式：YYYY-MM）进行分组统计。SQL 查询条件为 `user_id = #{userId} AND lottery_month = #{lotteryMonth} AND is_deleted = 0`，通过 `COUNT(*)` 统计该用户在本月的抽奖记录数量。剩余次数计算公式为 `remainingChances = Math.max(0, monthlyLimit - usedChances)`，确保不为负数。

```
// 统计本月已抽奖次数
String currentMonth = LocalDateTime.now().format(MONTH_FORMATTER);
Integer usedChances = lotteryRecordMapper.countByUserIdAndMonth(userId, currentMonth);

// 计算剩余次数
```

```
Integer remainingChances = Math.max(0, monthlyLimit - usedChances);
vo.setCanLottery(remainingChances > 0);
```

`doLottery` 方法实现抽奖流程，使用 `@Transactional` 保证原子性。首先调用 `getLotteryInfo` 检查抽奖资格，如果 `canLottery` 为 `false`，抛出异常 "本月抽奖次数已用完"。

`performLottery` 方法实现概率抽奖算法。根据会员等级从数据库查询抽奖规则，构建奖池。概率计算采用累积概率算法。生成 0-99 的随机数，遍历奖池中的奖品，累加每个奖品的概率值，当随机数小于累积概率时，判定中奖。算法确保概率总和为 100，且按概率从高到低匹配，保证概率分布的准确性。

```
private LotteryResult performLottery(Integer memberLevel, Long currentPoints) {
    LotteryPool pool = getLotteryPool(memberLevel);

    // 生成随机数 (0-99)
    int randomValue = random.nextInt(100);

    // 根据概率确定中奖类型 (累积概率算法)
    int cumulativeProbability = 0;
    for (LotteryPrize prize : pool.getPrizes()) {
        cumulativeProbability += prize.getProbability();
        if (randomValue < cumulativeProbability) {
            return processPrize(prize, currentPoints);
        }
    }

    // 默认返回没中奖
    return new LotteryResult(0, null, null, null, "很遗憾，未中奖");
}
```

`processPrize` 方法处理不同类型的奖品。奖品类型包括：0-没中奖、1-固定积分、2-积分翻倍。对于固定积分类型 (`prize_type = 1`)，直接使用规则中配置的 `prize_points` 作为奖励积分数量，构建抽奖结果对象。对于积分翻倍类型 (`prize_type = 2`)，需要根据用户当前可用积分计算奖励。如果用户当前可用积分为 0，则改为奖励固定 100 积分，避免翻倍无意义。如果用户有可用积分，则奖励积分数量等于当前可用积分数量，同时记录翻倍倍数 (2.0) 和翻倍前的积分数量，用于前端展示和历史记录。

如果中奖 (`pointsReward > 0`)，调用 `pointsService.addPoints` 发放积分。积分来源设置为 3 (行为积分)，积分有效期为 15 天。

3.2.2 营销系统积分计算的技术实现

1. 订单积分计算的规则匹配算法

`calculateOrderPoints` 方法计算订单应获得的积分，支持基础消费积分、促销积分和等级积分。首先查询用户会员等级，然后查询基础消费积分规则，SQL 查询条件包括 `rule_type = 0`（消费积分）、`rule_status = 1`（启用）、会员等级匹配、时间范围匹配，按优先级降序取第一条规则。如果找到规则，使用 `orderAmount.multiply(pointsRatio).setScale(0, RoundingMode.DOWN).longValue()` 计算基础积分，使用 `RoundingMode.DOWN` 向下取整。

```
// 查询基础消费积分规则

MarketingPointsRule baseRule =
marketingPointsRuleMapper.selectBaseConsumptionRule(memberLevel);

Long basePoints = 0L;

if (baseRule != null && baseRule.getPointsRatio() != null) {
    basePoints = orderAmount.multiply(baseRule.getPointsRatio())
        .setScale(0, RoundingMode.DOWN).longValue();
}
```

促销积分规则匹配使用 XML 映射文件实现动态 SQL。查询条件包括：

- 1) 节假日匹配: `holiday_start <= orderDate <= holiday_end`
- 2) 订单金额范围匹配: `min_order_amount <= orderAmount <= max_order_amount`
- 3) 特定商品匹配: 通过中间表 `marketing_points_rule_food` 实现一对多关系。查询时使用 `EXISTS` 子查询判断:
 - 若规则在中间表中无关联商品（`NOT EXISTS`），则适用于所有商品
 - 若规则在中间表中有关联商品（`EXISTS`），则检查订单商品ID是否在规则关联的商品列表中（`rf.food_id IN`（订单商品ID列表））
- 4) 会员等级匹配: `member_level` 为空或等于用户会员等级

查询结果按优先级降序排序（`ORDER BY priority DESC`）。若找到促销规则，使用 `basePoints * (multiplier - 1)` 计算额外积分，其中 `multiplier` 为促销倍数（如 2.0 表示双倍积分）。

```
// 查询促销积分规则

List<MarketingPointsRule> promotionRules =

marketingPointsRuleMapper.selectPromotionRules(
    orderDateLocal, orderAmount, foodIds, memberLevel);

Long promotionPoints = 0L;
```

```

if (!promotionRules.isEmpty()) {
    MarketingPointsRule topRule = promotionRules.get(0); // 取最高优先级规则
    if (topRule.getPointsMultiplier() != null && basePoints > 0) {
        BigDecimal basePointsDecimal = BigDecimal.valueOf(basePoints);
        promotionPoints = basePointsDecimal.multiply(
            topRule.getPointsMultiplier().subtract(BigDecimal.ONE))
            .setScale(0, RoundingMode.DOWN).longValue();
    }
}

```

```

<select id="selectPromotionRules" resultMap="BaseResultMap">
    SELECT DISTINCT r.* FROM marketing_points_rule r
    <where>
        r.rule_type = 1 AND r.rule_status = 1 AND r.is_deleted = 0
        AND (r.member_level IS NULL OR r.member_level = #{memberLevel})
        AND (r.start_time IS NULL OR r.start_time <= NOW())
        AND (r.end_time IS NULL OR r.end_time >= NOW())
        <if test="orderDate != null">
            AND (r.holiday_start IS NULL OR r.holiday_start <= #{orderDate})
            AND (r.holiday_end IS NULL OR r.holiday_end >= #{orderDate})
        </if>
        <if test="orderAmount != null">
            AND (r.min_order_amount IS NULL OR r.min_order_amount <=
                #{orderAmount})
            AND (r.max_order_amount IS NULL OR r.max_order_amount >=
                #{orderAmount})
        </if>
        <if test="foodIds != null and foodIds.size() > 0">
            AND (
                -- 规则没有指定商品（适用于所有商品）
                NOT EXISTS (
                    SELECT 1 FROM marketing_points_rule_food rf
                    WHERE rf.rule_id = r.id AND rf.is_deleted = 0
                )
            )
            OR
            -- 规则指定的商品在订单商品列表中
            EXISTS (
                SELECT 1 FROM marketing_points_rule_food rf
                WHERE rf.rule_id = r.id

```

```

        AND rf.is_deleted = 0
        AND rf.food_id IN
        <foreach collection="foodIds" item="foodId" open="(" separator="," close=")">
            #{foodId}
        </foreach>
    )
)
</if>
</where>
ORDER BY r.priority DESC
</select>

```

等级积分规则查询条件为 `rule_type = 2`（等级积分）、`member_level =` 目标等级，按优先级降序取第一条。如果找到规则，使用相同方式计算等级积分。最后将基础积分、促销积分、等级积分相加得到总积分，调用 `pointsService.addPoints` 发放积分，并更新订单的 `points_amount` 字段。

2.行为积分计算的实现方式

行为积分在用户执行特定行为时触发，包括点赞商家、收藏商家、还贷款和积分抽奖。积分抽奖作为行为积分的一种特殊形式独立于其他行为积分的计算，本点仅阐述其他三种行为积分的计算。

当用户点赞或收藏商家时，系统通过 `MerchantInteractionServiceImpl.updateInteraction` 处理用户与商家的互动状态更新，并在更新过程中判断是否触发行为积分奖励。系统首先查询用户对该商家的历史互动记录。如果记录不存在，创建新记录并将旧状态设为 `false`；如果记录存在，保存旧状态（`oldLiked`、`oldCollected`）用于判断是否为首次行为。系统通过比较新旧状态判断是否为首次行为：对于点赞，判断条件为 `dto.getLiked() != null && dto.getLiked() && (oldLiked == null || !oldLiked)`，即新状态为 `true` 且旧状态为 `false` 或 `null`；对于收藏，使用相同的逻辑判断。这种设计确保只有首次点赞或首次收藏才奖励积分，避免重复奖励。

```

// 判断是否是首次点赞（从 false 变为 true）
boolean shouldRewardLike = dto.getLiked() != null && dto.getLiked() &&
    (oldLiked == null || !oldLiked);
if (shouldRewardLike) {
    Long points = marketingPointsRuleService.calculateBehaviorPoints(dto.getUserId(), "like");
}

```

当用户还贷款时，系统通过 `WalletApplicationService.repayLoan` 处理还贷款操作。还贷款成功后，系统自动触发行为积分奖励。

如果判断为首次行为（点赞/收藏）或还贷款成功，系统调用 `MarketingPointsRuleServiceImpl.calculateBehaviorPoints` 计算应获得的积分。该方法首先查询用户的会员等级，然后根据行为类型和会员等级查询匹配的积分规则，使用 `marketingPointsRuleMapper.selectBehaviorRule` 方法查询。

规则查询的条件包括：`rule_type = 3`（行为积分）、`rule_status = 1`（启用）、`is_deleted = 0`（未删除）、`behavior_type = #{behaviorType}`（行为类型匹配）、`(member_level = #{memberLevel} OR member_level IS NULL)`（会员等级匹配或通用规则）、时间范围匹配（`start_time` 和 `end_time`）。这种设计使得每种行为类型都可以配置独立的积分规则，支持不同会员等级获得不同积分。

如果找到匹配的规则，系统从规则中获取 `points_amount` 字段作为奖励积分数量。行为积分使用固定积分数量，而不是按比例计算。系统构建 `PointsAddDTO` 对象，设置用户ID、积分数量、积分来源为 3（行为积分）、关联规则ID、描述信息。过期时间根据规则中的 `expire_days` 字段计算，如果规则未设置有效期，则使用默认值 30 天。积分发放通过调用 `pointsService.addPoints` 方法实现，传入 `PointsAddDTO` 对象。

3.2.3 RabbitMQ 异步消息处理的实现

1. RabbitMQ 配置与消息序列化

`RabbitMQConfig` 配置类创建订单支付完成队列，使用 `new Queue(ORDER_PAID_QUEUE, true)` 创建持久化队列，`durable=true` 保证队列在 RabbitMQ 重启后不丢失。配置 `Jackson2JsonMessageConverter` 作为消息转换器，将 Java 对象序列化为 JSON 格式发送到队列，消费者接收时自动反序列化为 Java 对象。

```
@Bean
public MessageConverter messageConverter() {
    return new Jackson2JsonMessageConverter();
}

@Bean
public RabbitTemplate rabbitTemplate(ConnectionFactory connectionFactory) {
    RabbitTemplate template = new RabbitTemplate(connectionFactory);
    template.setMessageConverter(messageConverter());
    return template;
}
```

RabbitTemplate 配置消息转换器后，调用 `convertAndSend` 方法时自动将 `OrderPaidMessage` 对象序列化为 JSON。`OrderPaidMessage` 包含订单ID、用户ID、订单金额、订单日期、商品ID列表和商品详情列表，使用 `@JsonFormat` 注解格式化日期时间字段。

2.消息发送与幂等性保证

订单支付完成时，`OrderServiceImpl.setOrderState` 方法调用 `sendOrderPaidMessage` 发送消息。方法构建 `OrderPaidMessage` 对象，包含订单信息和商品详情（商品ID、价格、数量），然后调用 `orderMessageService.sendOrderPaidMessage` 发送到队列。

```
private void sendOrderPaidMessage(Long orderId, Order order) {
    try {
        OrderPaidMessage message = new OrderPaidMessage();
        message.setOrderId(orderId);
        message.setUserId(order.getCustomerId());
        message.setOrderAmount(order.getOrderTotal());
        message.setOrderDate(order.getOrderDate());
        // 查询订单商品详情
        List<OrderDetail> orderDetails = orderDetailMapper.selectByOrderId(orderId);
        List<Long> foodIds = new ArrayList<>();
        List<OrderPaidMessage.OrderFoodDetail> foodDetails = new ArrayList<>();

        for (OrderDetail detail : orderDetails) {
            foodIds.add(detail.getFoodId());
            OrderPaidMessage.OrderFoodDetail foodDetail = new
                OrderPaidMessage.OrderFoodDetail();
            foodDetail.setFoodId(detail.getFoodId());
            foodDetail.setFoodPrice(detail.getFoodPrice());
            foodDetail.setQuantity(detail.getQuantity());
            foodDetails.add(foodDetail);
        }

        message.setFoodIds(foodIds);
        message.setFoodDetails(foodDetails);
        orderMessageService.sendOrderPaidMessage(message);
    } catch (Exception e) {
        log.error("发送订单支付完成消息失败: orderId={}, ", orderId, e);
    }
}
```

3.消息监听与积分计算的幂等性处理

OrderPaidListener 使用 @RabbitListener(queues = "order.paid.queue") 监听队列消息。收到消息后，调用 marketingPointsRuleService.calculateOrderPoints 计算积分。方法内部使用订单ID和积分来源作为幂等性检查：查询 points_transaction 表，如果已存在 related_order_id = orderId AND points_source = 0 的记录，说明已处理过，直接返回，避免重复发放积分。

```
@RabbitListener(queues = "order.paid.queue")
public void handleOrderPaid(OrderPaidMessage message) {
    try {
        log.info("收到订单支付完成消息: orderId={}, userId={}, orderAmount={}",
            message.getOrderId(), message.getUserId(), message.getOrderAmount());

        // 调用营销系统计算积分
        marketingPointsRuleService.calculateOrderPoints(
            message.getUserId(),
            message.getOrderId(),
            message.getOrderAmount(),
            message.getOrderDate(),
            message.getFoodDetails()
        );

        log.info("订单积分计算完成: orderId={}, userId={}",
            message.getOrderId(), message.getUserId());
    } catch (Exception e) {
        log.error("处理订单支付完成消息失败: orderId={}", message.getOrderId(), e);
        throw e; // 抛出异常，触发重试机制
    }
}
```

如果消息处理失败，抛出异常触发 RabbitMQ 重试机制。

3.2.4 Redis 缓存优化的实现

Redis 配置通过 RedisConfig 完成，使用 RedisTemplate<String, Object> 作为操作模板。序列化策略采用混合方式：Key 使用 StringRedisSerializer，Value 使用 Jackson2JsonRedisSerializer，支持复杂对象的序列化与反序列化。ObjectMapper 配置

为可见性 ANY，并启用默认类型信息，确保复杂对象（如 PointsAccountVO、List<PointsExpirationVO>）能正确序列化与反序列化。

系统采用 Cache-Aside（旁路缓存）模式。读操作先查缓存，命中直接返回；未命中查数据库，写入缓存后返回。写操作先更新数据库，成功后删除相关缓存，保证一致性。该模式将缓存作为数据源旁路，应用负责缓存与数据库的同步。

积分账户查询使用 Cache-Aside。getPointsAccount 先调用 pointsCacheService.getAccountCache(userId)，命中返回；未命中查询数据库，将 PointsAccount 转换为 PointsAccountVO（补充会员等级名称），调用 pointsCacheService.setAccountCache(userId, vo) 写入缓存。缓存 Key 为 points:account:{userId}，过期时间5分钟，平衡实时性与性能。

```
@Override
public PointsAccountVO getPointsAccount(Long userId) {
    // 1. 先查缓存
    PointsAccountVO cached = pointsCacheService.getAccountCache(userId);
    if (cached != null) {
        log.debug("从缓存获取积分账户，userId: {}", userId);
        return cached;
    }

    // 2. 缓存未命中，查数据库
    PointsAccount account = pointsAccountMapper.selectByUserId(userId);

    // ... 转换为 VO ...

    // 3. 写入缓存
    pointsCacheService.setAccountCache(userId, vo);
    return vo;
}
```

积分兑换比例查询也使用 Cache-Aside。calculateDeductibleAmount 先查缓存获取兑换比例；未命中查询数据库，获取 MarketingPointsExchangeRule 的 exchange_ratio，默认 100 积分=1 元，写入缓存。Key 为 points:exchange:ratio，过期时间 30 分钟，因变更频率低，可设置较长过期时间。

```
// 2. 获取积分兑换比例（使用缓存）
BigDecimal exchangeRatio = pointsCacheService.getExchangeRatioCache();
if (exchangeRatio == null) {
    // 缓存未命中，查数据库
```

```

MarketingPointsExchangeRule rule = exchangeRuleMapper.selectCashExchangeRule();
exchangeRatio = (rule != null && rule.getExchangeRatio() != null)
    ? rule.getExchangeRatio()
    : BigDecimal.valueOf(100); // 默认100积分=1元
// 写入缓存
pointsCacheService.setExchangeRatioCache(exchangeRatio);
}

```

即将过期积分列表查询同样使用 Cache-Aside。getExpiringPoints 先查缓存；未命中查询数据库，转换为 PointsExpirationVO 列表，写入缓存。Key 为 points:expiring:{userId}，过期时间 5 分钟，与积分账户缓存一致。

缓存更新采用删除策略。所有修改积分账户的操作（增加、扣减、冻结、解冻、升级会员等级、扣除冻结积分）完成后，删除相关缓存，确保下次查询获取最新数据。

缓存 Key 设计采用命名空间前缀，便于管理和批量操作。points:account:{userId} 用于积分账户，points:exchange:ratio 用于兑换比例，points:expiring:{userId} 用于即将过期积分。这种设计便于按前缀批量删除，也便于在 Redis 中查看和管理。

3.2.5 积分过期处理与预警的定时任务实现

1. 定时任务配置与执行机制

PointsExpirationSchedule 使用 Spring 的 @Scheduled 注解配置定时任务。积分过期处理任务每天凌晨 1 点执行（cron 表达式：0 00 01 * * ?），预警任务每天上午 9 点执行（cron 表达式：0 00 09 * * ?）。

```

@Scheduled(cron = "0 00 01 * * ?")
public void processExpiredPoints() {
    log.info("定时任务：开始处理积分过期");
    try {
        pointsExpirationService.processExpiredPoints();
        log.info("定时任务：积分过期处理完成");
    } catch (Exception e) {
        log.error("定时任务：积分过期处理失败", e);
    }
}

```

2. 积分过期处理的实现

processExpiredPoints 方法处理已过期的积分。首先查询所有已过期但未标记的积分记录，SQL 查询条件为 expire_time < #{currentTime} AND is_expired = 0。使用

`LocalDateTime.now()` 获取当前时间，确保使用应用服务器的时区（东八区），避免时区问题。

遍历过期记录，对每条记录：标记为已过期（`is_expired = 1`），查询原始交易记录获取积分来源等信息，使用 `selectForUpdate` 加行锁查询积分账户，计算新的可用积分和总积分，验证数据完整性（如果为负数抛出异常），更新积分账户，插入过期交易记录（`transaction_type = 2`）。

3.积分过期预警的实现

`checkAndSendExpirationAlerts` 方法实现预警逻辑。首先查询预警配置，检查是否启用。计算预警日期：`LocalDate.now().plusDays(config.getAlertDays())`，如果配置预警天数为 7，则查询 7 天后过期的积分。

查询即将过期的积分记录，SQL 查询条件为 `expire_date = #{alertDate} AND is_expired = 0`，按用户分组统计。遍历查询结果，对每个用户检查是否需要发送预警：查询预警日志表，如果不存在记录，说明首次预警，需要发送；如果存在记录，检查 `alert_cycle` 配置：如果为 `NULL`，表示只预警一次，已有记录则不发送；如果不为 `NULL`，检查 `next_alert_time`，如果当前时间大于等于下次预警时间，则发送预警。

```
// 检查是否需要发送预警
PointsExpirationAlertLog existingLog = alertLogMapper.selectByUserIdAndExpireDate(
    exp.getUserId(), alertDate);

boolean shouldSend = false;
if (existingLog == null) {
    // 第一次预警
    shouldSend = true;
} else {
    if (config.getAlertCycle() == null) {
        // alert_cycle 为 null 表示只预警一次
        shouldSend = false;
    } else if (existingLog.getNextAlertTime() != null &&
        existingLog.getNextAlertTime().isBefore(LocalDate.now())) {
        // 到了下次预警时间
        shouldSend = true;
    }
}
```

如果需要发送预警，调用 `sendAlert` 方法。方法获取用户信息（姓名、手机号），使用模板替换生成通知内容（支持变量：`{username}`、`{points}`、`{expireDate}`），创建通知记录（`notification_type = 2`，`audit_result = NULL`），插入预警日志记录，计算下次预警时间（如果 `alert_cycle` 不为 `NULL`，则为当前时间加预警周期），最后通过 `WebSocket` 推送给用户。

3.2.6 数据完整性保障的多层验证机制

1. 数据验证方法的实现

`validateAndSetAccountValues` 方法实现三层验证。第一层验证：检查总积分、可用积分、冻结积分是否小于 0，如果为负数，记录错误日志（包含用户ID、账户ID、异常值），抛出 `APIException`，错误码为 `POINTS_DATA_INCONSISTENT`，错误信息包含具体数值。

第二层验证：检查业务逻辑，验证总积分是否等于可用积分加冻结积分。如果不相等，记录错误日志（包含所有相关值），抛出异常。

第三层：验证通过后设置值。所有修改积分账户的操作都调用此方法，确保数据一致性。

2. 事务管理与并发控制机制

所有修改积分账户的方法都使用 `@Transactional` 注解，保证操作的原子性。如果方法执行过程中抛出异常，`Spring` 自动回滚事务，保证数据一致性。

关键操作使用数据库行锁防止并发问题。`selectForUpdate` 方法在查询时加行锁，SQL 语句为 `SELECT * FROM points_account WHERE user_id = #{userId} AND is_deleted = 0 FOR UPDATE`。`FOR UPDATE` 子句在事务提交前锁定该行，其他事务无法修改，防止并发修改导致的数据不一致。

例如，两个用户同时支付订单使用积分时，第一个事务加锁查询积分账户，第二个事务等待；第一个事务完成并提交后，第二个事务才能继续，保证积分扣减的准确性。

积分系统后端通过分层架构、依赖注入、接口编程实现模块化和可维护性。使用 `RabbitMQ` 实现异步消息处理，解耦订单系统和营销系统，提升系统性能和可扩展性。使用 `Redis` 缓存优化查询性能，采用 `Cache-Aside` 模式保证数据一致性。使用定时任务处理积分过期和预警，使用数据库行锁和事务管理保证并发安全。通过多层验证机制保障数据完整性，遵循 `Fail-Fast` 原则及时发现问题。整个系统设计遵循单一职责、依赖倒置、开闭原则等设计原则，符合“高内聚低耦合”的设计思想，代码结构清晰，易于维护和扩展。

第四章 模块测试

4.1 测试概览

本次测试旨在验证饿了么项目中积分系统功能的正确性、稳定性和安全性，确保系统满足需求文档中的所有功能要求。

4.1.1 测试工具

- Apifox（API测试）
- JUnit 5和 Spring Boot Test（后端测试）
- Chrome DevTools（前端调试）
- Vue DevTools（Vue调试）

4.1.2 测试范围

- 前端功能测试
- 后端单元测试
- 后端集成测试
- 后端API测试
- 前后端联合测试¹

¹ 管理端测试账号: 用户名 admin, 密码 Admin123

4.2 前端测试

4.2.1 页面加载与渲染测试（顾客端）

表 4-1 顾客端页面加载与渲染测试

测试项	测试步骤	预期结果	测试结果	备注
积分主页加载	1. 用户登录后，点击“我的”页面中的“我的积分”入口。 2. 观察页面加载过程。	1. 页面快速加载，无长时间白屏。 2. 正确显示积分总额、可用积分、冻结积分及会员等级。 3. 清晰展示“抽奖入口”、“即将过期积分”、“可兑换商品”等核心功能入口。	通过	页面加载时间符合性能要求。
积分明细页面加载与筛选	1. 进入“积分明细”页面。 2. 默认查看所有记录。 3. 分别选择“收入”、“支出”、“过期”等交易类型进行筛选。 4. 选择不同的积分来源进行筛选。	1. 列表按时间倒序正确加载所有积分变动记录。 2. 筛选后，列表动态刷新，仅显示符合筛选条件的记录。 3. 列表项信息完整（时间、类型、数量、余额、描述）。	通过	筛选功能响应迅速，数据准确。
促销规则页面加载	1. 从首页轮播图或积分主页进入“积分促销活动”页面。	1. 页面展示所有当前有效的积分获取促销规则。 2. 规则信息清晰（如：节假日双倍、满额奖励、指定商品等）。 3. 若规则关联指定商品，显示“立即购买”入口。	通过	规则信息与后台配置一致。

4.2.2 促销商品功能测试

表 4-2 促销商品功能测试

测试项	测试步骤	预期结果	测试结果	备注
促销入口跳转	1. 在首页看到带有“指定商品多倍积分”的促销轮播图。 2. 点击轮播图上的“立即购买”按钮。	1. 正确跳转至“促销指定商品”专属页面。 2. 页面内陈列所有参与该促销活动的商品。	通过	跳转逻辑正确，页面定位准确。
促销商品展示与购买	1. 在促销商品页面，浏览商品列表。 2. 选择任一商品，点击“购买”。	1. 商品信息展示完整（商品图、名称、价格、促销积分倍数提示）。 2. 点击后进入该商品的订单确认与支付页面，流程与普通商品一致。	通过	购买流程顺畅，积分倍数信息在支付页面有明确提示。

4.2.3 积分抵扣支付功能测试

表 4-3 积分抵扣支付功能测试

测试项	测试步骤	预期结果	测试结果	备注
支付页面积分抵扣展示	1. 用户有可用积分，提交一笔普通商品订单，进入支付页面。	1. 支付方式中，“使用积分抵扣”选项默认被勾选 2. 实时展示“可用积分总数”、“本次最多可抵扣XX元”、“抵扣后需支付XX元”等信息。	通过	抵扣金额计算准确，与后端接口返回一致。
积分抵扣开关操作	1. 在支付页面，手动取消勾选“使用积分抵扣”复选框。	1. 积分抵扣相关信息隐藏，待支付金额恢复为订单原价。	通过	交互响应即时。
积分不足提示	1. 用户可用积分不足以抵扣任何金额（如积分低于兑换门槛）。	1. “使用积分抵扣”选项显示为不可用（灰色）或明确提示“积分不足”。	通过	提示友好，避免用户误解。
支付成功结果页	1. 使用积分+现金混合支付成功	1. 支付成功页面明确展示“积分抵扣：-XX元”以及“现金支付：XX元”的明细。	通过	支付结果清晰透明

4.2.4 积分兑换商品功能测试

表 4-4 积分兑换商品功能测试

测试项	测试步骤	预期结果	测试结果	备注
积分商城浏览	1. 进入“积分商城”或“兑换商品”页面。	1. 列表展示所有可兑换商品，包含商品图、名称、所需积分、库存状态。 2. 积分不足的商品有置灰或“积分不足”提示。	通过	界面直观，库存状态实时更新。
商品兑换流程	1. 选择有库存且积分足够的商品，点击“立即兑换”。 2. 在兑换确认页选择配送地址并提交。	1. 进入兑换确认页，再次展示商品信息、所需积分和地址选择框。 2. 提交后，弹出“兑换成功”提示，积分余额立即减少。 3. 生成一条积分兑换订单，可在订单中心查看。	通过	兑换过程不经购物车，流程简洁。
兑换失败场景	1. 尝试兑换库存为0的商品。 2. 尝试兑换所需积分高于当前可用积分的商品。	1. 点击“兑换”时，弹出明确提示“商品库存不足”。 2. 按钮置灰或点击后提示“积分不足”。	通过	错误提示准确，阻止无效操作。
兑换订单不可取消	1. 在订单中心找到已兑换的积分订单，尝试取消。	1. 订单详情页和列表页均不显示“取消订单”按钮，或点击取消时提示“积分兑换订单不可取消”。	通过	符合业务规则，前端严格限制。

4.2.5 积分抽奖功能测试

表 4-5 积分抽奖功能测试

测试项	测试步骤	预期结果	测试结果	备注
抽奖页面信息展示	1. 不同等级会员（白银、黄金、钻石）登录，进入积分抽奖页面。	1. 页面中央显示抽奖转盘或类似组件。 2. 页面清晰展示“本月剩余抽奖次数：X次”，次数与会员等级对应（如白银1次）。 3. 显示“抽奖规则”入口。	通过	次数信息由后端接口正确返回并显示。

执行抽奖	1. 用户有剩余次数，点击“开始抽奖”按钮。	1. 触发抽奖动画。 2. 动画结束后，弹出弹窗显示抽奖结果（如“恭喜获得XX积分”、“很遗憾，未中奖”）。 3. 若中奖，积分余额实时更新。 4. 剩余抽奖次数立即减1。	通过	动画流畅，结果反馈及时，积分变动实时。
次数用尽提示	1. 用户本月抽奖次数已用完，再次点击“开始抽奖”。	1. 弹出提示“本月抽奖次数已用完，请升级会员获取更多机会”或类似信息。 2. 按钮可能变为“前往升级”或置灰。	通过	引导用户进行会员升级，转化路径清晰。
抽奖记录查询	1. 在抽奖页面点击“我的抽奖记录”。	1. 进入记录页面，列表按时间倒序展示历史抽奖结果。	通过	记录信息完整，包含时间、奖品内容

4.2.6 页面加载与渲染测试（管理端）

表 4-6 管理端页面加载与渲染测试

测试项	测试步骤	预期结果	测试结果	备注
营销中心 主页加载	1. 管理员登录管理后台，进入“营销中心”-“积分管理”。	1. 页面正确加载，展示当前生效的核心积分规则概览。 2. 清晰显示“规则管理”、“预警配置”、“兑换管理”三个功能入口。	通过	页面布局清晰，导航明确。
规则管理 列表加载	1. 进入“积分规则管理”页面。 2. 切换不同的规则类型（基础、促销、行为、等级）选项卡。	1. 列表按默认顺序（如优先级、创建时间）正确加载。 2. 切换选项卡时，列表内容动态更新为对应类型的规则。 3. 列表显示规则的名称、状态、优先级、生效时间等关键信息。	通过	列表支持分页，加载流畅。

4.2.7 规则管理功能测试

表 4-7 规则管理功能测试

测试项	测试步骤	预期结果	测试结果	备注
规则增删改查	1. 在规则管理列表页，点击“新增规则”。 2. 填写表单并提交。 3. 对已有规则进行“编辑”和“删除”操作。	1. 弹出新增/编辑表单，字段完整（规则名、类型、比例/倍数、会员等级、优先级、生效时间等）。 2. 提交后，列表刷新，新规则出现或旧规则更新。 3. 删除规则时，有二次确认弹窗，删除后列表更新。	通过	表单验证完整（如数值范围、时间逻辑），操作即时生效。
规则状态切换	1. 找到一条启用中的规则，点击“禁用”按钮	1. 规则状态立即变为“禁用”，该规则在前端立即失效。	通过	状态切换无需刷新页面，体验流畅。
规则搜索与排序	1. 在搜索框输入规则名称关键词。 2. 点击表头按“优先级”或“创建时间”排序。	1. 列表动态过滤，只显示名称包含关键词的规则。 2. 列表顺序根据点击的列进行升序/降序排列。	通过	搜索和排序功能响应迅速。

4.2.8 预警配置功能测试（管理端）

表 4-8 管理端预警配置功能测试

测试项	测试步骤	预期结果	测试结果	备注
查看与修改预警配置	1. 进入“积分到期预警配置”页面。	1. 页面显示当前配置，如“提前7天预警”、“预警周期：每天”。 2. 管理员可以修改“提前天数”和“预警周期”，并保存。	通过	配置项清晰，保存后提示成功。

4.2.9 兑换管理功能测试（管理端）

表 4-9 兑换管理功能测试

测试项	测试步骤	预期结果	测试结果	备注
管理可兑换商品	1. 在“积分兑换管理”页面，进行商品的“上架”、“下架”、“编辑”操作 2. 编辑商品信息，如所需积分、库存数量。	1. 列表展示所有已上架和已下架的兑换商品。 2. 上架/下架操作后，商品在积分商城立即显示/隐藏。 3. 修改库存后，用户端兑换时立即遵循新库存规则	通过	管理操作与用户端状态实时同步，确保了业务一致性。

4.3 后端测试

4.3.1 单元测试

1. 积分账户操作单元测试

对 `PointsServiceImpl` 的单元测试使用 Mock 隔离依赖。使用 `@MockBean` 模拟 `PointsAccountMapper`、`PointsTransactionMapper`、`PointsExpirationMapper`、`PointsCacheService` 等，专注验证业务逻辑。

测试 `addPoints` 时，覆盖订单奖励积分冻结场景。Mock `selectForUpdate` 返回账户对象，调用 `addPoints` 后验证：总积分和冻结积分增加，可用积分不变；插入的 `PointsTransaction` 的 `transaction_type=0`、`points_source=0`；创建了 `PointsExpiration` 记录；调用了 `deleteUserAllCache` 删除缓存。使用 `ArgumentCaptor` 捕获插入参数，验证字段值。

测试 `freezePoints` 时，验证积分+现金支付场景。Mock 账户有足够可用积分，调用后验证：可用积分减少，冻结积分增加，总积分不变；插入的 `PointsTransaction` 的 `transaction_type=3`、`points_source=5`、`points_change` 为负数；调用了 `deleteAccountCache`。同时测试积分不足时抛出 `APIException`。

测试 `deductPoints` 时，验证优先扣减即将过期积分。Mock 多条过期记录，调用后验证：按过期时间升序扣减；过期记录的 `points_amount` 正确减少；完全扣减的记录被标记为 `is_expired=true`；插入的消费明细正确。

2. 积分规则计算单元测试

对 `MarketingPointsRuleServiceImpl` 的单元测试，使用 Mock 隔离数据库和积分服务。

测试 `calculateOrderPoints` 时，模拟多种促销规则（节假日双倍、订单金额阈值、特定商品），验证规则匹配与积分计算。Mock `selectBaseConsumptionRule`、`selectPromotionRules` 返回规则列表，调用后验证：基础积分正确计算；促销积分按

规则叠加；总积分正确；调用了 `pointsService.addPoints` 且参数正确；更新了订单的 `points_amount`。

测试 `calculateBehaviorPoints` 时，验证会员等级匹配。Mock 不同会员等级的规则，调用后验证：优先匹配精确等级规则；无精确规则时匹配通用规则；规则未找到时返回 0；调用了 `pointsService.addPoints` 且 `points_source=3`。

3. 积分抽奖服务单元测试

对 `PointsLotteryServiceImpl` 的单元测试，使用 Mock 隔离数据库和积分服务。

测试 `getLotteryInfo` 时，验证抽奖信息查询逻辑。Mock `pointsAccountMapper.selectByUserId` 返回不同会员等级的账户，Mock `lotteryRecordMapper.countByUserIdAndMonth` 返回已抽奖次数，调用后验证：会员等级正确识别；每月抽奖次数限制根据会员等级正确计算；剩余次数正确计算；`canLottery` 标志正确设置；奖池配置根据会员等级正确返回。

测试 `doLottery` 时，验证抽奖流程。Mock `getLotteryInfo` 返回有剩余次数，Mock `pointsAccountMapper.selectForUpdate` 返回账户，Mock `lotteryRuleMapper.selectByMemberLevel` 返回抽奖规则列表，调用后验证：抽奖记录正确插入（包含用户ID、会员等级、抽奖类型、月份等）；如果中奖，调用 `pointsService.addPoints` 且 `points_source=3`、`expireTime` 为15天后；抽奖记录的 `transactionId` 正确更新；剩余次数正确更新。同时测试次数用完时抛出 `APIException("LOTTERY_NO_CHANCE")`。

测试 `performLottery` 时，验证概率算法。Mock 不同会员等级的规则列表，使用固定随机数种子，调用后验证：概率分布正确（累积概率算法）；固定积分类型正确返回积分数量；积分翻倍类型正确计算奖励；用户无积分时翻倍改为固定100积分；没中奖类型返回0积分。

测试 `getLotteryRecords` 时，验证记录查询。Mock `lotteryRecordMapper.selectRecentByUserId` 返回记录列表，调用后验证：返回记录数量不超过限制；记录按时间降序排序；记录包含完整的抽奖信息。

4. 缓存服务单元测试

对 `PointsCacheServiceImpl` 的单元测试，使用 `@MockBean` 模拟 `RedisTemplate`。

测试缓存读写时，验证序列化与反序列化。Mock `redisTemplate.opsForValue().get` 返回序列化对象，调用 `getAccountCache` 后验证：正确转换为 `PointsAccountVO`；缓存未命中返回 `null`。调用 `setAccountCache` 后验证：调用了 `redisTemplate.opsForValue().set`；设置了正确的过期时间。

测试异常处理时，验证缓存失败不影响业务。Mock `redisTemplate` 抛出异常，调用缓存方法后验证：捕获异常并记录日志；返回 `null` 或忽略，不抛出异常。

4.3.2 集成测试

1. 积分账户操作集成测试

使用 @SpringBootTest 加载完整 Spring 上下文，使用真实数据库。使用 @Transactional 和 @Rollback 保证测试后回滚。

表 4-10 积分账户操作集成测试

测试项	请求参数	预期结果	实际结果	测试结果
增加积分（行为积分）	userId=100, points=100, pointsSource=3	总积分和可用积分增加100，生成积分明细记录	总积分和可用积分正确增加，明细记录正确生成	通过
增加积分（订单奖励）	userId=100, points=200, pointsSource=0, relatedOrderId=1000	总积分和冻结积分增加200，可用积分不变	总积分和冻结积分正确增加，可用积分不变	通过
冻结积分（积分+现金支付）	userId=100, points=500, orderId=1000	可用积分减少500，冻结积分增加500，总积分不变	可用积分和冻结积分正确变化，总积分不变	通过
冻结积分（积分不足）	userId=100, points=10000, orderId=1000	抛出APIException，提示积分不足	抛出异常，错误信息正确	通过
解冻积分（订单取消）	userId=100, orderId=1000	冻结积分转回可用积分，总积分不变	积分正确解冻，总积分不变	通过
解冻奖励积分（订单完成）	userId=100, orderId=1000	冻结的奖励积分转为可用积分	奖励积分正确解冻	通过
扣减积分（优先过期）	userId=100, points=300	优先扣减即将过期的积分，过期记录更新	按过期时间优先扣减，记录正确更新	通过
扣除冻结积分（订单完成）	userId=100, orderId=1000	总积分和冻结积分减少，生成消费明细	积分正确扣除，消费明细正确生成	通过
查询积分账户	userId=100	返回积分账户信息（总积分、可用积分、冻结积分、会员等级）	返回正确的积分账户信息	通过
查询积分账户（缓存命中）	userId=100（第二次查询）	从缓存返回积分账户信息	从缓存返回，响应时间更快	通过
查询积分明细（分页）	userId=100, pageNum=1, pageSize=10	返回第一页的10条积分明细	返回正确的分页结果	通过
查询积分明细（筛选）	userId=100, transactionType=0, pointsSource=0	返回筛选后的积分明细	返回正确的筛选结果	通过

升级会员等级	userId=100, newMemberLevel=1	会员等级更新为白银, 获得等级积分	会员等级和积分正确更新	通过
数据完整性验证	执行多次积分操作后	totalPoints = availablePoints + frozenPoints	数据一致性验证通过	通过
负数保护	尝试扣减超过可用积分的积分	抛出APIException, 提示数据异常	抛出异常, 数据未损坏	通过

2. RabbitMQ 消息处理集成测试

表 4-11 RabbitMQ 消息处理集成测试

测试项	请求参数	预期结果	实际结果	测试结果
订单支付消息发送	订单支付后发送 OrderPaidMessage	消息成功发送到RabbitMQ队列	消息成功发送, 队列中有消息	通过
订单支付消息消费	监听器接收 OrderPaidMessage	OrderPaidListener接收消息并触发积分计算	监听器正确接收消息, 积分计算触发	通过
积分计算触发	订单金额200元, 用户为黄金会员	根据规则计算并发放积分	积分正确计算并发放	通过
消息消费幂等性	重复发送相同订单的支付消息	相同订单只计算一次积分	幂等性验证通过, 不会重复计算	通过
消息消费失败处理	发送格式错误的消息	消息消费失败, 记录错误日志	错误被正确处理, 不影响其他消息	通过

3. Redis 缓存集成测试

表 4-12 Redis 缓存集成测试

测试项	请求参数	预期结果	实际结果	测试结果
缓存写入	查询积分账户后	积分账户信息写入Redis缓存	缓存成功写入, Key为points:account:100	通过
缓存读取	第二次查询积分账户	从缓存读取积分账户信息	从缓存返回, 响应时间减少	通过
缓存过期	等待5分钟后查询	缓存过期, 从数据库重新查询	缓存过期后重新查询数据库	通过

缓存删除	增加积分后	相关缓存被删除	缓存成功删除，下次查询从数据库读取	通过
兑换比例缓存	查询兑换比例	兑换比例写入缓存，30分钟过期	缓存成功写入，过期时间正确	通过
即将过期积分缓存	查询即将过期积分	即将过期积分列表写入缓存	缓存成功写入，数据正确	通过
缓存异常处理	Redis服务不可用时	降级到直接查询数据库，不抛出异常	系统正常运行，从数据库查询	通过
批量删除缓存	执行积分操作后	用户相关的所有缓存被删除	积分账户和即将过期积分缓存都被删除	通过

4. 积分规则计算集成测试

表 4-13 积分规则计算集成测试

测试项	请求参数	预期结果	实际结果	测试结果
基础消费积分计算	订单金额100元，基础比例1%	获得1积分	积分正确计算并发放	通过
促销积分计算（双倍）	订单金额100元，节假日双倍规则	获得2积分	促销规则正确应用	通过
促销积分计算（订单金额阈值）	订单金额300元，满200双倍规则	获得6积分（ $300 \times 1\% \times 2$ ）	订单金额阈值规则正确应用	通过
促销积分计算（特定商品）	订单包含特定商品，双倍积分规则	特定商品获得双倍积分	商品特定规则正确应用	通过
会员等级积分差异	普通用户和黄金会员分别下单	黄金会员获得更多积分	会员等级规则正确应用	通过
行为积分计算（点赞）	用户首次点赞商家	根据会员等级获得行为积分	行为积分正确发放	通过
行为积分计算（收藏）	用户首次收藏商家	根据会员等级获得行为积分	行为积分正确发放	通过
行为积分计算（重复行为）	用户再次点赞同一商家	不发放积分	重复行为不奖励积分	通过
等级积分发放	用户升级为白银会员	获得白银会员等级积分	等级积分正确发放	通过

5. 积分兑换集成测试

表 4-14 积分兑换集成测试

测试项	请求参数	预期结果	实际结果	测试结果
积分兑换商品（成功）	foodId=1, addressId=1, userId=100	积分扣减， 创建积分兑换订单	积分正确扣减， 订单正确创建	通过
积分兑换商品（积分不足）	foodId=1, addressId=1, userId=100（积分不足）	返回错误， 提示积分不足	返回400错误， 提示信息正确	通过
积分兑换商品（商品不存在）	foodId=999, addressId=1, userId=100	返回错误， 提示商品不存在	返回400错误， 提示信息正确	通过
积分兑换订单不可取消	创建积分兑换 订单后尝试取消	订单状态不允许取消	取消操作被拒绝	通过
积分+现金支付	订单金额100元， 使用积分抵扣	积分冻结， 订单金额减少	积分正确冻结， 订单金额正确计算	通过
积分+现金支付（订单完成）	订单完成后	冻结积分被扣除， 奖励积分解冻	积分状态正确更新	通过
积分+现金支付（订单取消）	订单取消后	冻结积分解冻， 奖励积分取消	积分状态正确恢复	通过
可抵扣金额计算	订单金额100元， 可用积分5000	可抵扣50元 (5000/100)	可抵扣金额正确计算	通过
可抵扣金额计算（超过订单金额）	订单金额50元， 可用积分10000	可抵扣50元 (不超过订单金额)	可抵扣金额不超过订单金额	通过

6.积分抽奖集成测试

表 4-15 积分抽奖集成测试

测试项	请求参数	预期结果	实际结果	测试结果
获取抽奖信息（白银会员）	userId=100（白银会员）	返回抽奖信息：会员等级=1，每月限制=1次，剩余次数=1，奖池配置正确	返回正确的抽奖信息	通过
获取抽奖信息（黄金会员）	userId=100（黄金会员）	返回抽奖信息：会员等级=2，每月限制=2次，剩余次数=2	返回正确的抽奖信息	通过
获取抽奖信息（钻石会员）	userId=100（钻石会员）	返回抽奖信息：会员等级=3，每月限制=3次，剩余次数=3	返回正确的抽奖信息	通过
获取抽奖信息（普通用户）	userId=100（普通用户）	返回抽奖信息：会员等级=0，每月限制=0次，canLottery=false	普通用户无抽奖机会	通过
执行抽奖（中奖-固定积分）	userId=100（白银会员，有剩余次数）	抽奖成功，获得固定积分，积分账户增加，生成抽奖记录和积分明细	抽奖成功，积分正确增加	通过
执行抽奖（中奖-积分翻倍）	userId=100（黄金会员，有剩余次数，当前积分500）	抽奖成功，获得500积分（翻倍），积分账户增加，生成抽奖记录和积分明细	积分翻倍正确计算	通过
执行抽奖（没中奖）	userId=100（有剩余次数）	抽奖成功，未获得积分，生成抽奖记录（pointsReward=0）	没中奖正确处理	通过
执行抽奖（次数用完）	userId=100（本月已抽完）	抛出APIException，提示"本月抽奖次数已用完"	次数限制正确生效	通过
执行抽奖（积分翻倍-无积分）	userId=100（无可用积分，抽中翻倍）	抽奖成功，获得固定100积分（翻倍改为固定积分）	无积分时翻倍正确处理	通过
每月次数限制	userId=100（白银会员），连续抽奖2次	第一次成功，第二次抛出异常	每月次数限制正确	通过
跨月次数重置	userId=100（上月已抽完），本月查询	剩余次数重置为每月限制次数	跨月次数正确重置	通过

查询抽奖记录	userId=100, limit=10	返回最近10条抽奖记录，按时间降序	返回正确的记录列表	通过
抽奖记录关联	userId=100（中奖后）	抽奖记录的transactionId正确关联积分明细	记录关联正确	通过
会员等级变更影响	userId=100（从白银升级到黄金）	每月抽奖次数限制从1次变为2次	会员等级变更正确影响抽奖限制	通过
奖池配置查询	userId=100（不同会员等级）	返回对应会员等级的奖池配置（奖品类型、概率、描述）	奖池配置正确返回	通过
事务回滚	抽奖过程中发生异常	抽奖记录和积分操作回滚，数据保持一致	事务回滚正确	通过

7. 积分过期处理集成测试

表 4-16 积分过期处理集成测试

测试项	请求参数	预期结果	实际结果	测试结果
查询即将过期积分	userId=100	返回即将过期的积分列表（按过期时间升序）	返回正确的过期积分列表	通过
查询已过期积分	userId=100	返回已过期的积分列表	返回正确的已过期积分列表	通过
统计即将过期积分	userId=100	返回即将过期的积分总数	返回正确的积分总数	通过
处理过期积分	定时任务执行	过期积分被标记为已过期，总积分和可用积分减少	过期积分正确处理，账户余额正确更新	通过
过期积分预警	积分即将过期时	发送预警通知	预警通知正确发送	通过
过期积分预警（单次）	alertCycle=null	只发送一次预警	预警只发送一次	通过
过期积分预警（循环）	alertCycle=7	每7天发送一次预警	预警按周期发送	通过

4.3.3 API测试

1. 用户端积分系统 API 测试

表 4-17 用户端积分系统 API 测试

测试项	请求路径与参数	预期结果	实际结果	测试结果
查询积分账户 (成功)	GET /api/points/account, 已认证用户	HTTP 200 , 返回积分账户信息	返回正确的积分账户数据	通过
查询积分账户 (未认证)	GET /api/points/account, 未认证	HTTP 401	返回401未授权错误	通过
查询积分账户 (缓存命中)	GET /api/points/account, 第二次查询	HTTP 200 , 从缓存返回	响应时间更快, 数据正确	通过
查询积分明细 (分页)	GET/api/points/transactions?pageNum=1&pageSize=10	HTTP 200 , 返回分页结果	返回正确的分页数据	通过
查询积分明细 (按类型筛选)	GET/api/points/transactions?transactionType=0&pointsSource=0	HTTP 200 , 返回筛选结果	返回正确的筛选数据	通过
积分兑换商品 (成功)	POST /api/points/exchange, {"foodId":1,"addressId":1}	HTTP 200 , 返回订单ID	积分扣减, 订单创建成功	通过
积分兑换商品 (积分不足)	POST /api/points/exchange, 积分不足	HTTP 400 , 错误提示	返回400错误, 提示信息正确	通过
获取可兑换商品列表	GET /api/points/exchange-goods	HTTP 200 , 返回商品列表	返回商品信息和所需积分	通过
获取积分兑换比例	GET /api/points/exchange-ratio	HTTP 200 , 返回兑换比例	返回正确的兑换比例	通过
计算可抵扣金额 (有积分)	GET /api/points/deductible-amount?orderAmount=100	HTTP 200 , 返回可抵扣金额	返回正确的可抵扣金额	通过
计算可抵扣金额 (无积分)	GET /api/points/deductible-amount?orderAmount=100, 无积分	HTTP 200 , 返回0	返回0	通过
查询即将过期积分	GET /api/points/expiring	HTTP 200 , 返回即将过期积分列表	返回正确的过期积分列表	通过

查询已过期积分	GET /api/points/expired	HTTP 200 ， 返回已过期积分列表	返回正确的已过期积分列表	通过
统计即将过期积分	GET /api/points/expiring/count	HTTP 200 ， 返回积分总数	返回正确的积分总数	通过
查询抽奖信息（成功）	GET /api/points/lottery/info, 已认证用户（会员）	HTTP 200 ， 返回抽奖信息	返回正确的抽奖信息	通过
执行抽奖（成功）	POST /api/points/lottery/draw, 已认证用户（有剩余次数）	HTTP 200 ， 返回抽奖结果	抽奖成功，返回正确结果	通过
查询抽奖记录（成功）	GET /api/points/lottery/records?limit=10	HTTP 200 ， 返回抽奖记录列表	返回正确的记录列表	通过

2. 管理员端营销系统 API 测试

表 4-18 管理员端营销系统 API 测试

测试项	请求路径与参数	预期结果	实际结果	测试结果
创建积分规则（成功）	POST /api/marketing/points/rules, 管理员权限	HTTP 200 ， 返回规则ID	规则创建成功，数据库记录正确	通过
创建积分规则（权限不足）	POST /api/marketing/points/rules, 普通用户	HTTP 403	返回403禁止访问	通过
更新积分规则（成功）	PUT /api/marketing/points/rules/{id} ， 规则DTO	HTTP 200 ， 返回true	规则更新成功，数据库记录更新	通过
删除积分规则（成功）	DELETE /api/marketing/points/rules/{id}	HTTP 200 ， 返回true	规则逻辑删除成功	通过
查询积分规则列表	GET/api/marketing/points/rules?pageNum=1&pageSize=10&ruleType=1	HTTP 200 ， 返回规则列表	返回正确的分页和筛选结果	通过
创建积分兑换规则（成功）	POST /api/marketing/points/exchange-rules ， 兑换规则DTO	HTTP 200 ， 返回规则ID	兑换规则创建成功	通过

更新积分兑换规则（成功）	PUT /api/marketing/points/exchange-rules/{id}	HTTP 200 ， 返回 true	兑换规则更新成功	通过
删除积分兑换规则（成功）	DELETE /api/marketing/points/exchange-rules/{id}	HTTP 200 ， 返回 true	兑换规则删除成功	通过
查询积分兑换规则列表	GET /api/marketing/points/exchange-rules?pageNum=1&pageSize=10	HTTP 200 ， 返回规则列表	返回正确的分页结果	通过
查询预警配置	GET /api/marketing/points/alert-config	HTTP 200 ， 返回预警配置	返回正确的预警配置信息	通过
更新预警配置（成功）	PUT /api/marketing/points/alert-config, 配置对象	HTTP 200 ， 返回 true	预警配置更新成功	通过

4.3.4 数据完整性测试

表 4-19 数据完整性测试

测试项	请求参数	预期结果	实际结果	测试结果
数据一致性验证	执行多次积分操作后查询账户	totalPoints = availablePoints + frozenPoints	数据一致性验证通过	通过
负数保护（扣减）	尝试扣减超过可用积分的积分	抛出APIException，提示数据异常	抛出异常，数据未损坏	通过
负数保护（冻结）	尝试冻结超过可用积分的积分	抛出APIException，提示积分不足	抛出异常，数据未损坏	通过
负数保护（解冻）	尝试解冻超过冻结积分的积分	抛出APIException，提示数据异常	抛出异常，数据未损坏	通过
过期积分处理	积分过期后查询账户	过期积分被扣除，账户余额正确	过期积分正确处理，余额正确	通过
事务一致性	积分操作过程中发生异常	所有操作回滚，数据保持一致	事务回滚成功，数据一致	通过

4.4 前后端联合测试

4.4.1 用户积分全流程测试

测试目的：验证用户从登录到完成积分相关操作的全流程功能完整性，确保前后端数据同步与用户体验流畅。

表 4-20 用户积分全流程测试

序号	测试场景	前端操作	后端预期行为	预期结果	实际结果	测试结果
1	用户登录并查看积分	1. 用户使用有效账号密码登录 2. 进入"我的积分"页面	1. 验证用户身份 2. 查询用户积分账户信息并返回	1. 登录成功 2. 页面正确显示总积分、可用积分、会员等级	登录成功，积分信息显示正确	通过
2	查看积分明细	1. 点击"积分明细" 2. 筛选"冻结中"类型记录	1. 接收查询请求 2. 按条件查询数据库并返回结果	1. 进入明细页面 2. 列表仅显示冻结中类积分记录	筛选功能正常，数据与后端一致	通过
3	浏览可兑换商品	1. 进入"我的积分"页面的"积分兑换商品"部分 2. 查看商品详情	1. 查询可兑换商品列表 2. 返回商品信息和所需积分	1. 展示所有可兑换商品 2. 显示库存和所需积分	商品列表加载正常，信息完整	通过
4	积分兑换商品	1. 选择有库存商品点击"兑换" 2. 选择配送地址并确认	1. 验证用户积分和商品库存 2. 扣除积分，创建兑换订单	1. 进入兑换确认页 2. 兑换成功，积分减少，生成订单	兑换流程完整，积分实时扣减	通过
5	查看兑换订单	1. 进入"我的订单" 2. 查看积分兑换订单	1. 查询用户订单列表 2. 返回订单详情	1. 订单列表中包含兑换订单 2. 订单状态为"已完成"	订单信息显示正确，状态准确	通过

4.4.2 积分支付流程完整测试

表 4-21 积分支付流程完整测试

步骤	操作	预期结果	实际结果	测试结果
1	用户提交订单进入支付页	支付页面显示订单总价100元，用户有5000积分	页面显示正确	通过
2	系统自动计算积分抵扣	显示"本次最多可抵扣50元"，"抵扣后需支付50元"	抵扣金额计算正确	通过
3	用户取消积分抵扣勾选	待支付金额恢复为100元，积分抵扣信息隐藏	交互响应正常	通过
4	用户重新勾选积分抵扣并确认支付	调用支付接口，积分冻结，支付流程继续	积分冻结成功，支付继续	通过
5	支付成功回调处理	1. 积分从冻结转为扣除 2. 订单状态更新为"已支付"	积分状态正确更新，订单状态正确	通过
6	支付成功页面展示	显示"积分抵扣：-50元"，"现金支付：50元"	支付明细展示清晰	通过
7	返回积分主页查看	积分余额减少50元（5000-50=4950）	积分余额更新正确	通过

4.4.3 订单积分奖励流程测试

表 4-22 订单积分奖励流程测试

步骤	操作	预期结果	实际结果	测试结果
1	黄金会员支付200元订单	订单支付成功	支付成功	通过
2	RabbitMQ发送支付消息	消息队列接收到OrderPaidMessage	消息发送成功	通过
3	营销系统监听消息	OrderPaidListener接收到消息并开始处理	监听器正常工作	通过
4	计算订单积分	根据黄金会员1:1比例，计算基础积分200分	基础积分计算正确	通过
5	检查促销规则匹配	检查是否有适用的促销规则（如节假日双倍）	规则匹配逻辑正确	通过
6	发放积分到用户账户	积分增加到用户账户（冻结状态）	积分增加成功，状态为冻结	通过
7	前端积分余额更新	用户积分总额增加，但可用积分不变（冻结中）	前端显示正确	通过

8	订单完成后解冻积分	订单状态变为"完成", 冻结积分转为可用	积分解冻成功	通过
---	-----------	----------------------	--------	----

4.4.4 积分抽奖全流程测试

表 4-23 积分抽奖全流程测试

步骤	操作	预期结果	实际结果	测试结果
1	钻石会员进入抽奖页面	显示"本月剩余抽奖次数: 3次"	次数显示正确	通过
2	点击"开始抽奖"按钮	触发抽奖动画, 发送抽奖请求	动画流畅, 请求发送	通过
3	后端处理抽奖逻辑	1. 检查剩余次数 2. 根据概率算法确定奖项 3. 发放积分奖励	抽奖逻辑执行正确	通过
4	前端显示抽奖结果	弹出弹窗显示"恭喜获得200积分"	结果反馈及时	通过
5	积分余额实时更新	用户积分总额立即增加200分	积分实时更新	通过
6	剩余次数减少	显示"本月剩余抽奖次数: 2次"	次数更新正确	通过
7	查看抽奖记录	在记录页面看到刚才的抽奖记录	记录保存完整	通过
8	抽奖次数用尽后再试	提示"本月抽奖次数已用完"	限制生效, 提示友好	通过

4.4.5 促销规则动态展示测试

表 4-24 促销规则动态展示测试

测试场景	前端操作	后端配置	预期结果	测试结果
节假日促销	用户在国庆期间访问首页	后台配置"十一黄金周双倍积分"规则	首页轮播图展示国庆双倍积分活动	通过
指定商品促销	用户点击轮播图"立即购买"	规则关联特定商品ID列表	跳转到指定商品列表页面	通过
会员专属促销	钻石会员浏览促销页面	配置"钻石会员满200双倍积分"规则	页面展示会员专属优惠提示	通过

规则过期处理	活动结束后用户查看	规则结束时间已过，状态为禁用	页面不再显示已过期的促销活动	通过
--------	-----------	----------------	----------------	----

4.4.6 积分过期预警流程测试

表 4-25 积分过期预警流程测试

步骤	操作	预期结果	实际结果	测试结果
1	管理员配置预警规则	设置"提前3天预警"，"每天提醒"	配置保存成功	通过
2	用户有积分即将过期	用户有100积分将在3天后过期	系统检测到即将过期积分	通过
3	定时任务触发预警	每天上午9点执行预警检查	任务按时执行	通过
4	生成预警消息	创建"您的100积分将于3天后过期"消息	消息内容正确	通过
5	前端消息中心展示	用户消息中心显示积分过期提醒	消息推送成功	通过
6	用户查看即将过期积分	点击积分页面"即将过期"入口	显示即将过期积分列表	通过
7	积分过期后自动清理	过期当天凌晨1点执行清理任务	过期积分被扣除	通过

4.4.7 管理端规则配置同步测试

表 4-26 管理端规则配置同步测试

测试项	管理员操作	用户端效果	预期结果	测试结果
新增基础积分规则	添加"铂金会员消费1元得2积分"	铂金会员下单后获得2倍积分	规则立即生效	通过
修改兑换比例	将兑换比例从100:1改为50:1	用户支付时可抵扣金额翻倍	比例实时更新	通过
禁用促销规则	禁用"双十二双倍积分"规则	用户下单不再享受双倍积分	规则立即失效	通过
上架新兑换商品	添加新商品"咖啡券，需5000积分"	积分商城显示新商品可兑换	商品实时同步	通过
调整抽奖概率	修改白银会员中奖概率分布	白银会员抽奖结果分布改变	概率调整生效	通过

测试结论：通过全面的前后端联合测试，积分系统所有核心功能均正常工作，前后端数据同步准确，用户体验流畅。

第五章 问题与解决&结果反思

5.1 问题与解决

下面主要列出一下较为重要的问题，在实现过程的debug中还遇到许多细节的问题与bug，主要与编写失误与逻辑疏忽有关，不一一列出。

1.循环依赖问题

初始运行后出现循环依赖。原因是 PointsService 依赖 MarketingPointsExchangeRuleService，而 MarketingPointsExchangeRuleService 又依赖 PointsService，形成了循环。

解决：在 PointsServiceImpl 中移除对 MarketingPointsExchangeRuleService 的依赖，改为直接注入 MarketingPointsExchangeRuleMapper，在 calculateDeductibleAmount 中直接调用 exchangeRuleMapper.selectCashExchangeRule()。这样打破循环，保持功能不变，符合依赖倒置原则。

2.数据库约束违反问题

创建积分过期预警通知时出现 SQLIntegrityConstraintViolationException: Column 'audit_result' cannot be null。原因是 notification 表的 audit_result 字段不允许 NULL，但积分过期预警不需要审核结果。

```
2025-11-27 17:02:00.023 [scheduling-1] INFO com.tju.elm_bk.schedule.PointsExpirationSchedule - ===== 定时任务，开始检查并发送积分过期预警 =====
2025-11-27 17:02:00.024 [scheduling-1] INFO com.tju.elm_bk.schedule.PointsExpirationSchedule - 【定时任务】任务执行时间: 2025-11-27T17:02:00.024818300
2025-11-27 17:02:00.033 [scheduling-1] INFO com.zaxxer.hikari.HikariDataSource - HikariPool-1 - Starting...
2025-11-27 17:02:09.771 [scheduling-1] INFO com.zaxxer.hikari.pool.HikariPool - HikariPool-1 - Added connection com.mysql.cj.jdbc.ConnectionImpl@394cb39f
2025-11-27 17:02:09.773 [scheduling-1] INFO com.zaxxer.hikari.HikariDataSource - HikariPool-1 - Start completed.
2025-11-27 17:02:09.786 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - ===== 开始检查并发送积分过期预警 =====
2025-11-27 17:02:09.848 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【预警配置】配置ID: 1, 是否启用: true, 预警天数: 3, 预警周期: null天, 模板: string
2025-11-27 17:02:09.860 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【预警日期】当前日期: 2025-11-27, 预警天数: 3, 计算的预警日期: 2025-11-30
2025-11-27 17:02:09.860 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【查询结果】查询到 1 条即将在 2025-11-30 过期的积分记录
2025-11-27 17:02:09.861 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【处理用户】用户ID: 6, 积分数量: 62, 过期日期: 2025-11-30
2025-11-27 17:02:09.876 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【预警判断】用户ID: 6 - 首次预警(无历史记录) - 需要发送预警
2025-11-27 17:02:09.876 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【发送预警】用户ID: 6, 积分数量: 62, 过期日期: 2025-11-30, 原因: 首次预警(无历史记录)
2025-11-27 17:02:09.876 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【发送预警详情】开始为用户ID: 6 发送预警, 积分数量: 62, 过期日期: 2025-11-30
2025-11-27 17:02:09.886 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【用户信息】用户ID: 6, 姓名: 宋, 电话: 16607302890
2025-11-27 17:02:09.886 [scheduling-1] INFO c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【通知内容】用户ID: 6, 原始模板: string, 生成内容: string
2025-11-27 17:02:10.052 [scheduling-1] ERROR c.t.e.s.impl.PointsExpirationAlertServiceImpl - 【通知创建失败】用户ID: 6, 创建通知失败, 错误:
### Error updating database. Cause: java.sql.SQLIntegrityConstraintViolationException: Column 'audit_result' cannot be null
### The error may exist in file [C:\Users\Administrator\Desktop\elm\frontend-comprehension\elm_bk\target\classes\com\tju\elm_bk\mapper\VirtualWalletVipRuleMapper.xml]
### The error may involve defaultParameterMap
### The error occurred while setting parameters
### SQL: insert into notification (user_id, notification_type, notification_content, audit_result, is_read, create_time, read_time, is_deleted) values (?, ?, ?, ?, ?, ?, ?, ?)
### Cause: java.sql.SQLIntegrityConstraintViolationException: Column 'audit_result' cannot be null
; Column 'audit_result' cannot be null
org.springframework.dao.DataIntegrityViolationException:
### Error updating database. Cause: java.sql.SQLIntegrityConstraintViolationException: Column 'audit_result' cannot be null
### The error may exist in file [C:\Users\Administrator\Desktop\elm\frontend-comprehension\elm_bk\target\classes\com\tju\elm_bk\mapper\VirtualWalletVipRuleMapper.xml]
### The error may involve defaultParameterMap
### The error occurred while setting parameters
```

图 5-1 数据库约束违反问题

解决：修改 notification 表结构，将 audit_result 改为 tinyint NULL DEFAULT NULL，允许为 NULL。在 elm_v2.sql 中添加 ALTER TABLE 语句，并更新 Notification 实体类，使 auditResult 可为 null。这样既满足预警通知的需求，又保持其他通知类型的约束。

3.时区不一致问题

SQL 查询中使用 NOW() 返回的时间不是东八区，导致定时清除过期积分无法正确执行。原因是应用默认时区与数据库时区不一致。

解决：在 Java 代码中，将 LocalDateTime.now() 作为参数传递给 SQL，而不是在 SQL 中使用 NOW()。例如，在 PointsExpirationMapper 的 selectExpiringPoints 方法中，将 LocalDateTime currentTime 作为参数传入 SQL，确保使用应用时区的时间。这样保证时区一致。

4.业务逻辑漏洞造成用户可用积分数可能不一致问题

由于订单在支付时便在积分过期表中记录了数据，在每日的定时任务清理过期积分时，若用户订单支付后一直不完成，等该笔订单过期后在清理时会扣除该用户的可用积分数，但因为该订单未完成，相应的积分数仍处于冻结状态而非可用状态，扣除用户的可用积分数会造成积分数不一致。

解决：增加订单支付七天后状态自动变为完成，七天小于系统积分的最短有效期，可以保证不会出现上述问题。

5. 前端组件间状态同步困难问题

在实现积分支付、积分兑换等功能时，多个页面组件（如“我的积分”页、“支付”页、“积分商城”页）都需要实时显示用户的当前积分余额。最初采用各自独立调用后端接口的方式，导致用户在一个页面操作后，其他页面无法及时更新数据，需要手动刷新。

解决：引入全局事件总线（Event Bus）机制。在关键积分变动操作（如支付成功、兑换成功、抽奖成功）完成后，通过事件总线发布积分更新事件（如 points-updated）。所有关注积分余额的组件监听该事件，在事件触发时主动重新拉取最新数据。同时，在需要强一致性的场景（如支付页面），采用短轮询方式定期查询积分账户状态，确保数据同步。

6. 页面交互体验不一致问题

在积分抽奖、商品兑换等需要等待后端响应的操作中，由于网络延迟或服务器处理时间，用户点击后界面无即时反馈，容易导致用户重复点击。同时，不同页面的加载状态提示（如加载中、成功、失败）样式和逻辑不统一。

解决：统一封装异步操作交互组件。针对按钮类操作，封装带有防抖和 loading 状态的 AsyncButton 组件，在请求期间禁用按钮并显示加载动画。针对页面级操作，使用 Vue 的异步组件配合 Suspense 处理加载状态。同时，引入统一的 Toast 通知组件，用于显示操作成功或失败提示，并确保提示风格和时长一致。

7. 促销规则轮播图动态跳转逻辑复杂问题

首页促销规则轮播图需要支持点击后根据规则类型跳转到不同页面（如指定商品列表、会员升级页等），且跳转时需携带参数。最初采用硬编码的跳转逻辑，导致新增规则类型时代码需频繁修改。

解决：设计可配置的跳转映射表。后端在返回促销规则列表时，为每条规则增加 actionType 和 actionParam 字段，分别表示跳转类型（如 VIEW_FOOD_LIST、UPGRADE_MEMBER）和参数（如商品ID列表）。前端根据映射表将 actionType 解析为对应的路由路径和参数，实现动态路由跳转。这样新增规则类型时只需在后端配置，前端无需修改。

5.2 结果反思

在模块实现的代码过程中，尽量避免一次性写很多的功能接口，而要完成一小部分时及时进行测试，使得小问题能及时解决，否则随着代码量和逻辑联系的增加，识别问题所在的难度和时间也会增加，更容易出现大问题。

同时在实现时要多注意字段、属性等名称的对应，往往一些细小的问题会出现在名称错误上而非代码本身问题。在数据库操作中，删改操作一定要保证WHERE子句的正确性，否则会造成难以恢复的错误。

异步消息处理方面，RabbitMQ 的引入实现了订单系统与营销系统的解耦，提高了系统的响应速度和吞吐量。但也需要注意消息的幂等性处理，避免重复消费导致积分重复发放。通过使用订单ID作为幂等键，确保了消息消费的幂等性。

第六章 实践成果与总结

6.1 实践总结

本次综合实践在完成前后端分离（SpringBoot + VUE）项目基础流程的基础上，成功集成了核心营销模块——积分系统²。相较于传统简单的积分记录模式，我们实现了完整的积分生命周期管理，包括积分获取、冻结、解冻、扣减、过期处理等，显著提升了系统的业务复杂度和技术深度³。主要技术突破与学习体会如下：

(1) 分层架构与设计原则实践

在 Service 层严格遵循面向对象设计原则，构建了清晰的分层架构。将积分账户操作、积分规则计算、积分兑换等业务逻辑封装在对应的 Service 实现类中，通过接口定义契约，实现类实现接口，调用方依赖接口而非实现类。这一实践带来的优势：增强了代码的内聚性和可维护性，避免了业务逻辑在 Service 层过度膨胀，为复杂的积分业务逻辑提供了清晰且稳健的实现基础。通过单一职责原则、依赖倒置原则、开闭原则等的应用，使系统具有良好的可扩展性和可测试性。

(2) 积分操作的原子性保障

在实现积分增加、扣减、冻结、解冻等操作时，重点关注了操作原子性的保证。通过 @Transactional 注解和数据库行锁（SELECT ... FOR UPDATE），确保积分账户余额的更新、积分明细的插入、过期记录的创建等操作要么同时成功，要么同时失败，这是积分系统稳定运行的基石。特别是在并发场景下，通过行锁机制防止了积分重复计算或扣减的问题。

(3) 复杂业务逻辑与规则引擎实现

成功实现了多种积分获取渠道（消费积分、促销积分、等级积分、行为积分）、积分冻结/解冻机制（订单奖励积分冻结、积分+现金支付冻结）、积分过期处理、积分过期预警等高级功能。这些机制的实现，让我们深入理解了电商平台如何通过可配置规则实现灵活的营销策略。所有积分规则都存储在数据库中，管理员可以通过管理接口动态配置，无需修改代码，大大提高了系统的灵活性和可维护性。

(4) 异步消息处理与缓存优化

成功集成了 RabbitMQ 实现订单系统与营销系统的异步解耦，订单支付完成后通过消息队列异步触发积分计算，提高了系统的响应速度和吞吐量。同时，通过 Redis 缓存优化了积分账户查询、兑换比例查询、即将过期积分查询等高频操作，显著降低了数据库压力，提升了系统性能。缓存采用 Cache-Aside 模式，写操作后删除相关缓存，保证了数据一致性。

² 仓库链接: <https://gitee.com/dai-mingjing/frontend-comprehension.git>

³ 本项目已部署至云服务器，体验可访问网址 <http://bobchasm.cn:8081/>

(5) 数据完整性保护机制

实现了严格的积分账户数据完整性验证机制，通过 `validateAndSetAccountValues` 方法确保总积分、可用积分、冻结积分始终满足 $\text{totalPoints} = \text{availablePoints} + \text{frozenPoints}$ 的约束，并且任何积分值都不能为负数。当检测到数据异常时，采用 Fail-Fast 原则立即抛出异常，而不是自动修正，确保问题能够及时发现和解决，避免了错误数据的传播。

6.2 未来展望

本次实践虽已完成核心业务功能链的构建，但面对未来大规模用户带来的高并发挑战，系统仍需深度优化。下一阶段，我们将着重提升性能和架构的健壮性：

性能优化方面，通过引入压力测试来识别瓶颈，进一步优化 Redis 缓存策略，考虑使用分布式缓存和缓存预热机制。对于积分计算等耗时操作，可以探索使用消息队列的延迟消息功能，实现更精细的异步处理。同时，可以考虑对积分明细表进行分表处理，按用户ID或时间维度进行分表，提高大数据量下的查询性能。

架构演进方面，探索如微服务等架构演进方案，将积分系统、营销系统、订单系统等核心模块进行解耦，使系统更贴近实际生产应用。可以考虑将积分系统独立为微服务，通过 API 网关统一管理，实现更好的服务治理和监控。

功能扩展方面，可以增加积分转赠功能，允许用户之间转让积分；实现积分商城功能，提供更多积分兑换商品和优惠券；增加积分排行榜功能，激励用户积极使用积分；实现积分有效期提醒的多种方式，如短信、邮件、App推送等。

安全性方面，通过限制多端登录、增加操作日志审计、实现积分操作的风控规则（如单日积分获取上限、异常操作检测）等措施进一步提高系统安全性和容错性，确保系统在高负载下稳定运行，持续向贴近现实应用的复杂需求标准迈进。